Bachelor of Science in Electrical and Electronic Engineering

EEE 400 (July 2025): Thesis

PATHOK: A Bangla Handwritten and Scene Text Recognition Model using Grapheme Tokenization

Submitted by

Mohammed Irtisam Sajin

2006057

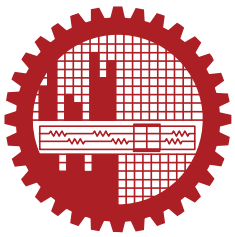
Md. Kaidul Islam

2006111

Supervised by

Dr. Maruf Ahmed

Assistant Professor



Department of Electrical and Electronic Engineering

Bangladesh University of Engineering and Technology

Dhaka, Bangladesh

June 2026

CANDIDATES' DECLARATION

This is to certify that the work presented in this thesis, titled, "PATHOK: A Bangla Handwritten and Scene Text Recognition Model using Grapheme Tokenization", is the outcome of the investigation and research carried out by us under the supervision of Dr. Maruf Ahmed.

It is also declared that neither this thesis nor any part thereof has been submitted anywhere else for the award of any degree, diploma or other qualifications.

Mohammed Irtisam Sajin
2006057

Md. Kaidul Islam
2006111

CERTIFICATION

This thesis titled, **"PATHOK: A Bangla Handwritten and Scene Text Recognition Model using Grapheme Tokenization"**, submitted by the group as mentioned below has been accepted as satisfactory in partial fulfillment of the requirements for EEE 400: Project/Thesis course, and as the requirements for the degree B.Sc. in Electrical and Electronic Engineering in June 2026.

Group Members:

Mohammed Irtisam Sajin

Md. Kaidul Islam

Supervisor:

Dr. Maruf Ahmed
Assistant Professor
Department of Electrical and Electronic Engineering
Bangladesh University of Engineering and Technology

Thesis Outcome Evidence and Engineering Complexity Justification

PATHOK: A Bangla Handwritten and Scene Text Recognition Model using Grapheme Tokenization

CO	Course Outcome Statement	Mapped PO	Evidence from Thesis Work	Thesis Reference	Attained?
CO1	Identify, formulate, research literature, interpret data, and analyze problems using principles of mathematical, natural and engineering sciences.	PO(b)	*Identified challenges in Bangla handwritten and scene text OCR through literature review and formulated the research problem. Analyzed research gaps in encoder and tokenizer selection for Bangla OCR.*	*Chapter: 1, 2* *Figure: 1.1*	☐
CO2	Investigate of problems using research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.	PO(d)	*Designed experiments using multiple encoder-tokenizer combinations and evaluated them using CER and WER. Performed comparative analysis and validated results on multiple datasets.*	*Chapter: 3, 4* *Figure: 3.1* *Table: 4.2 - 4.7*	☐
CO3	Apply knowledge of mathematics, science, and engineering to solve research problems	PO(a)	*Applied deep learning, computer vision, CNN, Vision Transformer, Transformer Decoder, and tokenization techniques to develop the OCR framework.*	*Chapter/Section: 3, 4.1* *Figure: 3.1 - 3.5*	☐
CO4	Demonstrate adherence to professional ethics and ethical principles of research	PO(h)	*Used publicly available datasets and properly cited all referenced works, datasets, and models. Ethical considerations and responsible use of OCR technology were discussed.*	*Section: 5.3*	☐
CO5	Communicate effectively with oral presentation and detailed thesis report	PO(j)	*Communicated research findings through structured thesis writing, figures, tables, experimental analysis, and result discussions.*	*Chapter/Section: All*	☐
CO6	Research contemporary aspects related to the problem and topic through detailed literature review and understand	PO(l)	*Reviewed recent OCR literature including CNN, CRNN, and Transformer-based approaches. Discussed research trends, limitations, and future research directions.*	*Chapter/Section: 2, 5.4, Reference*	☐

Knowledge Profile, Complex Engineering Problem, and Complex Engineering Activity Justification

Code	Meaning	Thesis-Specific Justification
Knowledge Profile Attributes		
K1	Knowledge of natural sciences applicable to the engineering discipline.	*The thesis utilizes concepts from digital image processing to analyze and recognize text from images. Understanding image characteristics such as noise, contrast, and illumination is important for OCR performance.*

Continued on next page

Code	Meaning	Thesis-Specific Justification
K2	Conceptually based mathematics, numerical analysis, statistics, and formal aspects of computer and information science supporting analysis and modelling.	*Mathematical concepts including matrix operations, probability distributions, optimization, attention mechanisms, and evaluation metrics such as Character Error Rate (CER) and Word Error Rate (WER) are used throughout the research. We have used tools like PyTorch, NumPy, Pandas, Kaggle etc. for performing our computations and validation tasks*
K3	Engineering fundamentals required for the engineering discipline.	*Core electrical and electronic engineering principles related to signal processing and computer vision are applied in developing the OCR framework.*
K4	Specialist knowledge that provides theoretical frameworks and bodies of knowledge for accepted practice areas in the engineering discipline.	*Specialized knowledge in Optical Character Recognition (OCR), Deep Learning, Convolutional Neural Networks (CNNs), Vision Transformers (ViTs), Transformer Decoders, and Bangla language tokenization techniques is employed in this work.*
K5	Engineering design knowledge that supports the design of complex engineering solutions.	*The thesis involves the design and implementation of an encoder-decoder OCR system. Different encoder and tokenizer combinations were evaluated and optimized to achieve improved recognition accuracy for Bangla handwritten and scene text.*
K6	Knowledge of engineering practice, including technology, tools, resources, safety, standards, and practical constraints.	*The research utilizes tools and resources including Python, PyTorch, deep learning libraries, publicly available datasets, and GPU-based model training and evaluation environments. Practical constraints such as dataset availability, computational resources, and model complexity were also considered.*
K7	Comprehension of the role of engineering in society, including ethics, sustainability, economy, environment, health, safety, and public welfare.	*The proposed OCR system contributes to document digitization, preservation of Bangla language resources, and improved information accessibility. Ethical, legal, societal, and sustainability aspects of OCR deployment have also been discussed.*
K8	Engagement with research literature in the discipline.	*The research is guided by an extensive review of recent journal articles, conference papers, benchmark datasets, and state-of-the-art OCR models, which informed the design choices, experimental setup, and analysis of results.*
Complex Engineering Problem Attributes		
P1	Depth of knowledge required: the problem cannot be solved without applying principles from mathematics, natural sciences, engineering fundamentals, and specialist engineering knowledge.	*Bangla OCR requires knowledge of image processing, deep learning, and Bangla script characteristics to design and effective and reliable OCR pipeline.*
P4	Familiarity of issues: the problem may include infrequently encountered, partially defined, emerging, or research-oriented issues.	*Bangla scene text recognition and grapheme-based tokenization remain relatively underexplored research areas. The work addresses challenges related to complex Bangla scripts and limited scene text resources.*
P5	Extent of applicable codes: the problem may require codes, standards, engineering practices, or literature that are not fully sufficient by themselves.	*The study relies on publicly available benchmark datasets, OCR evaluation metrics, and state-of-the-art deep learning architectures. Existing methods do not fully address the combined handwritten and scene text recognition problem.*
P7	Interdependence: the problem has multiple connected components, variables, sub-systems, or constraints that influence one another.	*The OCR system consists of interconnected components including image preprocessing, encoder selection, transformer decoding, tokenization, training strategy, and evaluation. Changes in one component directly affect overall performance.*
Complex Engineering Activity Attributes		
A1	Range of resources: the activity uses diverse resources such as people, equipment, software, data, literature, laboratory facilities, or engineering tools.	*The research utilizes multiple resources including benchmark datasets, research literature, GPU computing resources, Python, PyTorch, and deep learning frameworks.*
A2	Level of interaction: the activity requires interaction among tasks, tools, systems, people, or disciplines.	*Integrating the most suitable combinations of Image Preprocessing, Tokenizers and encoders provide the best results.*

Continued on next page

Code	Meaning	Thesis-Specific Justification
A3	Innovation: the activity involves creative use of engineering knowledge, adaptation of methods, new design choices, or non-routine problem solving.	*The thesis proposes and evaluates a grapheme-based OCR framework for Bangla text while comparing multiple encoder-tokenizer combinations and extending recognition to both handwritten and scene text domains.*
A4	Consequences for society and environment: the activity may have effects on health, safety, sustainability, environment, economy, or society.	*The proposed system can support document digitization, information accessibility, and preservation of Bangla linguistic resources. It may benefit education, archival systems, and digital transformation initiatives.*

I/We, the undersigned, pledge that I/we filled out the tables with consultation and feedback of my/our supervisor.

Mohammed Irtisam Sajin
ID: 2006057

Md. Kaidul Islam
ID: 2006111

The relevant tables are verified based on the thesis report, presentation, demonstration, viva voce, and assessment evidence. Any suggested modifications are reflected in the table.

Dr. Maruf Ahmed
Assistant Professor

ACKNOWLEDGEMENT

First and foremost, we express our sincere gratitude to Almighty Allah for granting us the strength, perseverance, and opportunity to successfully complete this thesis work.

We express our deepest appreciation to our respected supervisor, Dr. Maruf Ahmed, Assistant Professor, Department of Electrical and Electronic Engineering (EEE), Bangladesh University of Engineering and Technology (BUET), for his invaluable guidance, continuous encouragement, constructive feedback, and unwavering support throughout the course of this research. His expertise, patience, and mentorship have played a crucial role in shaping this work, and it has been a privilege to work under his supervision.

We would like to extend our heartfelt gratitude to our beloved parents. Their unconditional love, constant encouragement, countless sacrifices, and unwavering belief in us have been the driving force behind our academic journey.

Finally, we would like to thank everyone who directly or indirectly contributed to the completion of this research and supported us throughout this journey.

Dhaka
June 2026

Mohammed Irtisam Sajin

Md. Kaidul Islam

Contents

CANDIDATES' DECLARATION i

CERTIFICATION ii

Thesis Outcome Evidence and Engineering Complexity Justification iii

ACKNOWLEDGEMENT vi

List of Figures ix

List of Tables x

ABSTRACT xi

1 Introduction 1

1.1 Motivation . . . 1

1.2 Research Objectives . . . 2

2 Literature Review 3

2.1 Classical OCR and Statistical Recognition Approaches . . . 3

2.2 Deep Learning-Based Approaches . . . 3

2.3 Sequence Recognition using RNNs and CTC . . . 4

2.4 Transformer-Based Approaches . . . 4

2.5 Scene Text Recognition and Signboard Analysis . . . 5

2.6 Bangla Handwritten and Scene Text Datasets . . . 5

2.7 Research Gaps . . . 5

3 Methodology 7

3.1 Model Architecture . . . 7

3.2 Encoder-Decoder Models . . . 8

3.2.1 Convolutional Neural Network (ResNet18) . . . 8

3.2.2 Transformer . . . 9

3.2.3 Vision Transformer (ViT) . . . 11

3.2.4 Tokenizer . . . 12

4 Results and Discussion **13**
4.1 Experimental Setup . . . 13
4.1.1 Dataset and Splits . . . 13
4.1.2 Model Variants . . . 13
4.1.3 Evaluation Metrics . . . 14
4.2 Results . . . 14
4.2.1 Handwritten Text Recognition . . . 14
4.2.2 Scene Text Recognition . . . 16
4.3 Discussion . . . 17
4.3.1 Effect of the Encoder . . . 17
4.3.2 Effect of the Tokeniser . . . 17
4.3.3 Cross-Dataset Generalisation . . . 18
4.3.4 Error Analysis . . . 19
4.3.5 Scene Text Recognition . . . 19

5 Evaluation **21**
5.1 Assessment of Issues . . . 21
5.1.1 Societal Issues . . . 21
5.1.2 Health and Safety Issues . . . 21
5.1.3 Legal and Cultural Issues . . . 21
5.2 Evaluation of Environment and Sustainability . . . 21
5.3 Ethical Issues . . . 22
5.4 Future Works . . . 22

References **24**

A Codes **30**
A.1 Dataset loading, model training and testing . . . 30
A.2 Synthetic Scene text Generation . . . 30

List of Figures

1.1 (a) Breakdown of Bangla word into graphemes. Graphemes can be vowels, consonants or consonant conjuncts, sometimes also along with vowel or consonant diacritics (b) Sample images of Bangla Handwritten Words showing differences in the handwriting styles [1] (c) Sample images of Bangla Scene Texts [2] . . . 2

3.1 Model Architecture. Shows the entire pipeline, consisting of the encoder, decoder and tokenizer. . . . 7
3.2 Original ResNet-18 Architecture. [3] . . . 9
3.3 Transformer Architecture. [4] . . . 10
3.4 Multi-head Attention [4] . . . 11
3.5 Different types of tokenizers [5] . . . 12

List of Tables

4.1 Model configurations evaluated in this work. † ResNet-18 + Character uses an unfiltered dataset (71,099 samples) with a 70/15/15 split and is included for reference only; results are not directly comparable. 14

4.2 Recognition performance on the BnHTRD test set (6,614 samples). The best result in each column is **bold**. † Evaluated on a different test split (10,665 samples) and is not directly comparable. 15

4.3 Cross-dataset evaluation of TrOCR_Grapheme. The model is trained exclusively on BnHTRD and evaluated on both datasets without re-training or fine-tuning. 15

4.4 Comparison of selected models evaluated on the BnHTRD dataset. Dashes indicate metrics not reported in the respective work. † Evaluated on a different BnHTRD split (same-domain). ‡ GraDeT-HTR evaluates on the full BnHTRD word set. § Evaluated on a filtered subset of 66,133 samples (80/10/10 split). ¶Evaluated on a 500-sample validation draw. 15

4.5 Scene text recognition results on the Bangla scene text test set (1,074 samples) after each training phase. The model is initialised from the TrOCR_Grapheme handwritten checkpoint. The best result in each column is **bold**. 16

4.6 Encoder ablation with grapheme tokenisation held constant. 17

4.7 Tokeniser ablation with TrOCR ViT encoder held constant. 17

ABSTRACT

Optical Character Recognition (OCR) is a key technology for converting text contained in images into machine-readable digital text. It plays an important role in document digitization, information retrieval, assistive technologies, and intelligent computer vision systems. While significant progress has been made in OCR for widely used languages, Bangla OCR remains challenging due to the complex structure of the Bangla script, which includes consonants, vowels, modifiers, compound characters, and grapheme clusters. In addition, variations in handwriting styles and the presence of scene text captured under diverse real-world conditions further increase the difficulty of accurate text recognition. This research aims to develop a robust OCR framework capable of recognizing both Bangla handwritten text and Bangla scene text while investigating the effectiveness of different image encoders and tokenization strategies.

The proposed OCR pipeline follows an encoder-decoder architecture. Two image encoders, namely ResNet-18 and a Vision Transformer (ViT)-based TrOCR encoder, are evaluated for visual feature extraction. A Transformer decoder is employed to generate text sequences from the extracted features. To study the impact of linguistic representation, three tokenization approaches are compared: grapheme-based tokenization, character-level tokenization, and Byte Pair Encoding (BPE). Experiments are conducted on the BnHTRD handwritten Bangla word dataset and additional evaluations are performed on the IIIT-Indic Bengali dataset to assess cross-dataset generalization. For Bangla scene text recognition, a synthetic-to-real training strategy is adopted using both generated and real scene text images.

Experimental results demonstrate that the choice of both encoder and tokenizer significantly influences OCR performance. The TrOCR encoder consistently outperforms the ResNet-18 encoder, highlighting the effectiveness of Transformer-based visual representations for Bangla text recognition. Among the tokenization methods, grapheme-based tokenization achieves the best results, followed by character-level tokenization and BPE. The best-performing configuration, TrOCR with grapheme tokenization, achieves a Word Accuracy of 92.21%, a Character Error Rate (CER) of 2.52%, and a Word Error Rate (WER) of 7.79% on the BnHTRD test set. For Bangla scene text recognition, the proposed three-phase training strategy achieves a Word Accuracy of 72.25% and a CER of 11.61%.

The findings demonstrate that grapheme-aware modeling and Transformer-based encoders provide substantial advantages for Bangla OCR. The proposed framework contributes toward the development of effective OCR systems for low-resource languages and supports the broader goals of document digitization and language preservation. Although cross-dataset evaluation reveals limitations in generalization across unseen handwriting styles and domains, the results suggest promising directions for future work, including multilingual OCR, end-to-end text detection and recognition systems, and the creation of larger Bangla scene text datasets.

Keywords: Bangla OCR, Handwritten Text Recognition (HTR), Scene Text Recognition (STR), Transformer, TrOCR, ResNet-18, Grapheme Tokenization.

MAIN BODY

CHAPTER 1: INTRODUCTION

1.1 Motivation

Optical Character Recognition (OCR) is the task of converting text contained in images into machine-readable digital text. These text images can be from handwritten or printed documents or other sources. The need for OCR technology has grown substantially in the digital age due to the increasing volume of textual information stored in physical formats.

The impact of OCR can be observed in a wide range of fields and applications, and it has become an essential component of intelligent systems such as automatic number plate recognition, autonomous vehicles, visual assistance tools for visually impaired individuals, real-time translation applications, and large language model development. [6]. One of the most significant applications is the digitization of handwritten and printed documents such as books, prescriptions, legal documents etc.

The use of OCR began in the 1950s, when a small predefined set of characters could be detected from scanned documents. As technology improved over the years, the overall performance increased and more diverse text recognition capabilities have been achieved [7]. Traditional OCR approaches typically relied on handcrafted features combined with statistical classifiers. But the introduction of neural networks enabled the learning and capturing of complex features, which greatly improved the detection of texts. Convolutional neural networks (CNN), Recurrent neural networks (RNN), attention mechanism, Bidirectional Long Short-Term Memory(Bi-LSTM) and different variations of the transformer model have shown remarkable results in the field of OCR [8] [9].

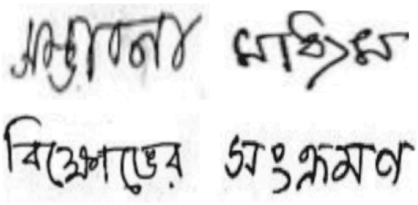
Bangla is among the most widely spoken languages in the world and is the national language of Bangladesh as well as a major language in eastern India. Large volumes of handwritten Bangla documents exist in educational institutions, government archives, and historical collections. So, there is a need for an OCR system that can reliably detect handwritten and printed Bangla text.

The challenge in detection of Bangla text lies within the complex nature of the language. Bangla words contain combinations of consonants, vowels and diacritics. Again, the words can be broken down into a set of graphemes. Graphemes are comprised of a vowel, consonant or consonant conjunct (grapheme root), sometimes paired together with vowel or consonant diacritics [10]. Fig. 1.1a shows how a bangla word can be broken down into individual graphemes. The vast amount of handwriting styles (Fig. 1.1b) also add to the complexity in detection.

Scene text refers to text captured in images or video frames (Fig. 1.1c). These can be images of texts in sign-boards, number plates, posters etc. Extracting these texts can be crucial in



(a)



(b)



(c)

Figure 1.1: (a) Breakdown of Bangla word into graphemes. Graphemes can be vowels, consonants or consonant conjuncts, sometimes also along with vowel or consonant diacritics (b) Sample images of Bangla Handwritten Words showing differences in the handwriting styles [1] (c) Sample images of Bangla Scene Texts [2]

detection of traffic violations, autonomous vehicles, computer vision and LLM applications etc. Scene texts pose challenges like alignment or orientations in addition to varying fonts and styles. Although progress has been made in OCR capabilities for Handwritten Bangla texts, detection of Bangla scene texts and design of a single model that can detect both, have been under-explored.

1.2 Research Objectives

The primary objective of this research is to develop a robust OCR pipeline capable of recognizing both handwritten and scene text from images, with a particular focus on Bangla text. The specific objectives of this research are as follows:

- Design and implementation of an OCR framework capable of processing images containing handwritten or scene texts.
- Analyzing the effectiveness of breaking down the detection of Bangla words into individual grapheme detection with suitable tokenizers.
- Evaluation of the proposed system using standard performance metrics such as Character Error Rate (CER) and comparison of the performance of different models.

CHAPTER 2: LITERATURE REVIEW

Bangla handwritten text recognition has been an active area of research due to the inherent complexity of the Bangla script. Unlike Latin-based scripts, Bangla contains a large number of basic and compound characters, as well as various modifiers and ligatures. The shapes of compound characters often differ significantly from their constituent basic characters, making recognition more challenging. Additionally, the presence of the *matra* (a horizontal line connecting characters) further complicates segmentation and character recognition.

2.1 Classical OCR and Statistical Recognition Approaches

Early studies on Bangla handwritten text recognition primarily relied on handcrafted feature extraction, segmentation-based optical character recognition (OCR), and classical machine learning algorithms. Basu et al. [11] proposed a hierarchical framework that combined segmentation and recognition. Their approach segmented word images based on Matra hierarchy and employed MLP classifiers along with topological features such as longest-run, modified shadow, and octant-centroid features. Das et al. [12] addressed the recognition of basic and compound characters using Multi-Layer Perceptron (MLP) and Support Vector Machine (SVM) classifiers. To overcome the challenges associated with character segmentation, Roy et al. [13] introduced a Hidden Markov Model (HMM)-based approach for offline Bangla handwritten word recognition. The proposed method divided word images into upper, middle, and lower zones and modeled the middle-zone components using HMMs and applying forced alignment for implicit character boundary detection. An improved variant further enhanced this framework using PHOG features and a water-reservoir-based refinement strategy, achieving better recognition performance on Bangla and similar scripts. [14]

2.2 Deep Learning-Based Approaches

With the advancement of deep learning, particularly Convolutional Neural Networks (CNNs), Bangla handwritten text recognition has achieved significant improvements over aforementioned traditional methods. CNN-based models learn hierarchical feature representations directly from images, making them more robust to writing variability and script complexity

Rabby et al. proposed BornoNet [15], a lightweight CNN model for recognizing 50 basic Bangla characters. Evaluated on multiple benchmark datasets such as CMATERdb [16], ISI [17], and BanglaLekha-Isolated [18], the model achieved strong performance despite its simple architecture, highlighting the potential of CNNs over traditional methods. EkushNet [19]

extended this idea by introducing a unified CNN framework capable of recognizing basic characters, digits, modifiers, and compound characters. Later works employed deeper CNN architectures(DCNNs) to improve feature representation and recognition accuracy, albeit with higher computational costs [20]. To further improve accuracy, ensemble and attention-based architectures were introduced. BanglaNet [21] combines multiple CNN backbones to enhance robustness through ensemble learning. Similarly, Khan et al. [22] proposed an SE-ResNeXt model that incorporates Squeeze-and-Excitation blocks for channel-wise feature recalibration, significantly improving the recognition of visually similar compound characters. In most cases with CNN-based detection characters are sequenced to make complete words. Maung et al. [23] implemented a spell checker with their CNN based architecture to further improve recognition accuracy.

2.3 Sequence Recognition using RNNs and CTC

CRNN-based approaches have been widely used for Bangla handwritten word recognition by treating the problem as sequence modeling rather than character segmentation. These methods typically combine a CNN for feature extraction, an RNN (LSTM/GRU) for sequence learning, and a CTC layer for alignment-free decoding.

Adak et al. [24] proposed an early CNN-LSTM-CTC model for cursive Bangla word recognition, where explicit segmentation was no longer needed. Safir et al. [25] improved this framework by exploring multiple CNN backbones with LSTM/GRU layers. They showed that stronger feature extractors reduce error rates. LILA-BOTI [26] further enhanced CRNN performance using knowledge distillation to improve recognition of rare and complex classes.

2.4 Transformer-Based Approaches

Recent advances in Bangla OCR and handwritten text recognition have increasingly adopted Transformer-based architectures due to their ability to model long-range contextual dependencies. Hasan et al. [5] proposed GraDeT-HTR, a resource-efficient Bengali handwritten text recognition system that combines a grapheme-based [10] tokenizer with a decoder-only Transformer. By leveraging grapheme-level representations , the model effectively captures the linguistic structure of Bangla while maintaining computational efficiency.

Ahmed et al. [27] introduced a hybrid Vision Transformer-based architecture for automated Bangla character recognition, achieving a validation accuracy of 95.80% across 84 character classes.Islam et al. [28] proposed BornoDrishti, a Vision Transformer-based framework. Similarly, Gupta et al. [29] developed CMViT OCR, which uses Vision Transformer components, demonstrating strong recognition performance on Bangla text along with other Indian-

languages.

2.5 Scene Text Recognition and Signboard Analysis

Scene text recognition focuses on extracting and recognizing text from natural images such as signboards and street scenes, where variations in lighting, background clutter, and perspective increase the difficulty of recognition.

Islam et al. [30] proposed a CNN-based framework for Bangla text extraction and character recognition from natural scene images. They later improved text localization through an efficient ROI detection algorithm, achieving more reliable recognition under challenging conditions [31]. Dutta et al. [32] introduced a multilingual scene text recognition system based on Faster R-CNN that integrates text detection, recognition, and text assembly for Bengali and English scripts. More recently, Mazumder et al. [33] developed an automated Bangla signboard analysis framework combining signboard detection, Tesseract-based text extraction, and signboard categorization for smart-city applications.

2.6 Bangla Handwritten and Scene Text Datasets

Bangla handwritten recognition research has been supported by datasets at varying levels of granularity. At the character level, CMATERdb [16], the ISI dataset [17], and Ekush [19] provide foundational benchmarks, while BanglaLekha-Isolated [18] remains the most comprehensive, covering 84 character classes across 166,105 images. At the word level, a handwritten Bengali word image dataset [34] provides 113,076 labelled samples across train, validation, and test splits, and BanglaWriting [35] offers 21,234 annotated words from 260 writers. For document-level tasks, BN-HTRd [1] is the largest available resource, with 788 full-page images from 150 writers containing over 108,000 word instances with line and word-level ground truth.

Scene text datasets for Bangla remain scarce. MLT-17 [36] and MLT-19 [37] include Bengali among multiple languages but consist predominantly of frontal, regular captures. IndicSTR12 [2] is a challenging resource available, providing 3,520 real Bengali word images from natural scenes under diverse conditions including blur, occlusion, and perspective distortion, alongside a large synthetic corpus.

2.7 Research Gaps

Although much work has been done on bangla OCR , transformer-based models for word level detection remains underexplored, as it requires large amounts of data [38]. Scene text detection is a relatively untouched problem in case of bangla script. Scene text detection will play a

significant role in drone-based monitoring and intelligent transportation systems [39]. To the best of our knowledge, no unified model has yet been proposed for both handwritten and scene text recognition in Bangla.

CHAPTER 3: METHODOLOGY

3.1 Model Architecture

Fig 3.1 shows the entire architecture of our proposed model. The pipeline starts with the image of the Bangla text, either handwritten or scene text, which is then preprocessed as required. All the images are resized to predefined height and width values. The preprocessed image is then fed as input to an image encoder. We have compared two different types of encoders, which has been discussed in later sections. The encoder extracts features from the text image and feeds them to the decoder. A transformer decoder is used which takes those encoder features and detects the tokens. The tokenizer then decodes the graphemes corresponding to those tokens and concatenates those graphemes to output the bangla text.

For training the model, the text label corresponding to the input image is used. The label text is first segmented into its graphemes and then tokenized. This is then used as ground truth to calculate the loss of the decoder output. Different tokenizer-encoder combinations were tested in terms of Character error rate (CER) and Word error rate (WER). These comparisons enabled us to propose our final model that can reliably detect Bangla texts over varying styles and modalities (handwritten or scene texts).

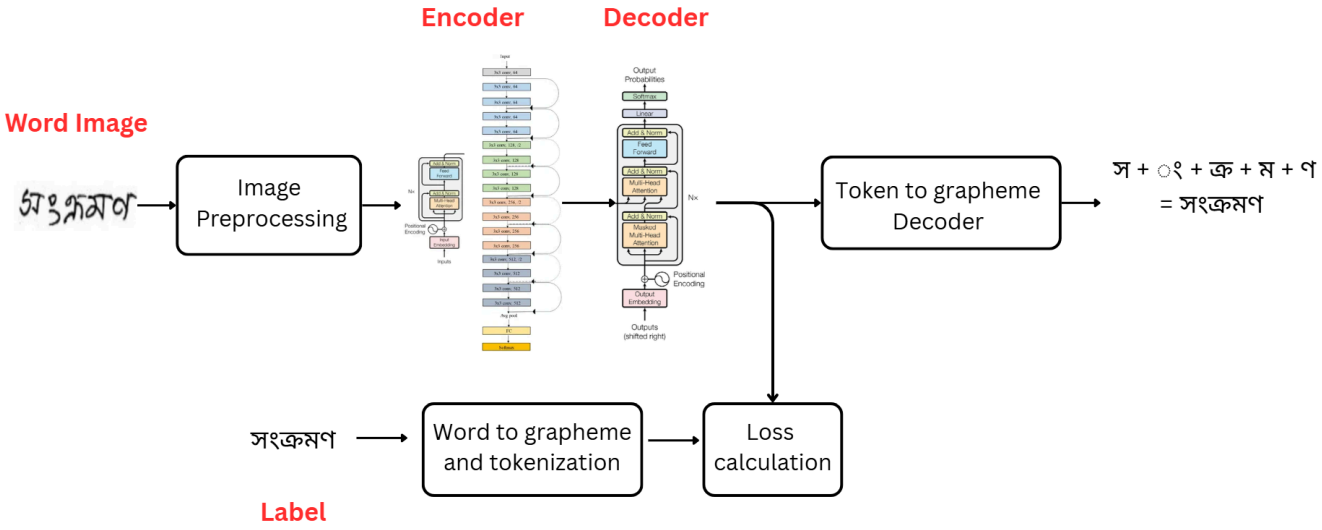


Figure 3.1: Model Architecture. Shows the entire pipeline, consisting of the encoder, decoder and tokenizer.

3.2 Encoder-Decoder Models

A suitable image encoder is a critical component of an OCR system because it is responsible for extracting meaningful visual features from input images before text recognition. A strong encoder can better capture character structures and contextual information, improving recognition accuracy under diverse and challenging conditions.

In this research, both CNN-based and Transformer-based encoders are investigated and compared to evaluate their effectiveness in feature representation. These models have been effective in capturing visual features from images of various domains over the years. Comparing these architectures enables a comprehensive assessment of their strengths and limitations, ultimately helping identify the most suitable encoder for robust Bangla text recognition.

The decoder on the other hand is responsible for converting the visual features extracted by the image encoder into meaningful text sequences. It can effectively learn the relationship between visual patterns and corresponding characters. Furthermore, the decoder captures contextual information between characters or graphemes and words, helping to reduce recognition errors and improve overall OCR performance.

3.2.1 Convolutional Neural Network (ResNet18)

Convolutional Neural Network, or CNN, is a form of Artificial Neural Network (ANN) that are primarily used for feature extraction in images. ANNs process each image pixel independently through fully connected layers, which leads to a large number of parameters and makes image processing computationally expensive. CNNs, on the other hand, are specifically designed for image data. They can automatically identify important visual features such as edges, shapes, and patterns by analyzing small regions of an image at a time. As a result, CNNs require fewer parameters, are less prone to overfitting, and generally achieve better performance in image-based tasks such as handwritten and scene text recognition [40].

ResNet (Residual Network) is a deep convolutional neural network architecture, introduced by [41], to address the challenges of training very deep neural networks. As the depth of a network increases, the training process becomes more difficult due to issues such as vanishing gradients [42] and degradation problems, where adding more layers can lead to higher training error and reduced accuracy.

To overcome these challenges, ResNet introduces skip connections (or shortcut connections) that allow information to bypass one or more layers and flow directly through the network. Instead of learning the complete mapping, the network learns only the residual information, making optimization easier and improving gradient flow during training. As a result, ResNet enables the successful training of much deeper networks while achieving higher accuracy and

better feature extraction capabilities for image-based tasks such as OCR.

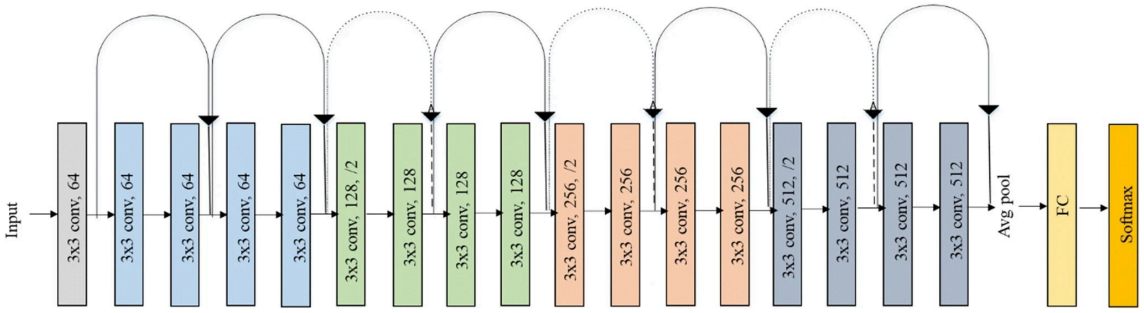


Figure 3.2: Original ResNet-18 Architecture. [3]

ResNet-18 [3] is a lightweight version of the Residual Network architecture that consists of 18 layers, including 17 convolutional layers and one fully connected layer. It uses residual (skip) connections to facilitate the training of deeper networks by improving information and gradient flow throughout the model. Compared to traditional CNNs, ResNet-18 can learn more discriminative features while maintaining relatively low computational complexity, making it a popular choice for image classification and feature extraction tasks.

3.2.2 Transformer

In 2017, transformer architecture was first introduced by Vaswani et al. [4]. Unlike traditional recurrent neural networks (RNNs) and convolutional neural networks (CNNs), transformers rely primarily on the attention mechanism to model relationships between different parts of the input data. This allows them to process all input elements simultaneously rather than sequentially, resulting in faster training and improved scalability.

The attention mechanism is the core component of the Transformer architecture. Its main purpose is to help the model focus on the most relevant parts of the input while processing a particular word, character, or image region. Instead of treating all input elements equally, attention assigns higher importance to the elements that are more useful for understanding the current context. This enables the model to capture long-range relationships and contextual information more effectively than traditional sequential models.

In a Transformer, each input is transformed into three vectors: *Query (Q)*, *Key (K)*, and *Value (V)*. The Query vector represents what information is being searched for, the Key vector represents the characteristics of each input element, and the Value vector contains the actual information to be passed forward. The similarity between Queries and Keys is used to determine how much attention should be given to each Value vector.

The Transformer uses Scaled Dot-Product Attention, which is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \tag{3.1}$$

where Q denotes the Query matrix, K denotes the Key matrix, V denotes the Value matrix, K^T represents the transpose of the Key matrix, d_k is the dimension of the Key vectors, and softmax$(\cdot)$ converts the similarity scores into attention weights.

The term QK^T computes similarity scores between Queries and Keys. Larger scores indicate stronger relationships between input elements. These scores are divided by $\sqrt{d_k}$ to prevent extremely large values that could make training unstable. The softmax function then converts the scores into probabilities, representing how much attention should be given to each input element. Finally, these attention weights are multiplied by the Value matrix V to generate a context-aware representation of the input.

This mechanism allows the Transformer to selectively focus on important information from different parts of the input sequence, enabling better understanding of contextual relationships. As a result, attention plays a crucial role in achieving high performance in tasks such as machine translation, image recognition, scene text recognition, and optical character recognition (OCR).

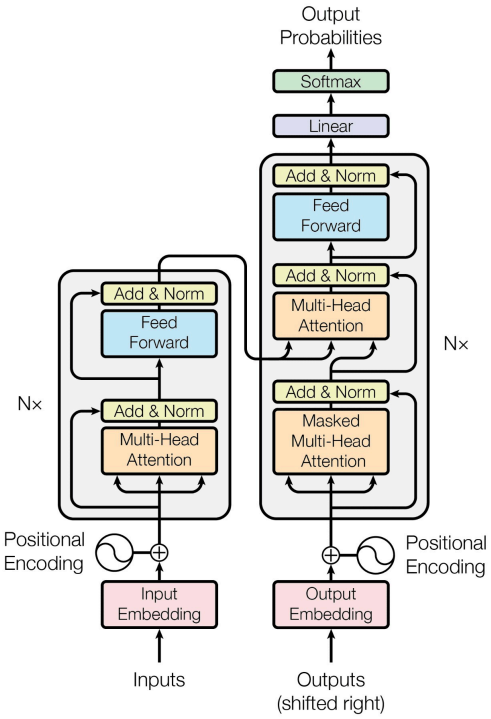


Figure 3.3: Transformer Architecture. [4]

Multi-Head Attention (Fig. 3.4) is an extension of the attention mechanism in which multiple attention operations are performed in parallel. Instead of learning a single set of relationships, each attention head focuses on different aspects of the input, allowing the model to capture diverse contextual information simultaneously. The outputs from all attention heads are then

concatenated and combined to form the final representation. This enables the Transformer to learn both local and global dependencies more effectively, leading to improved performance in complex tasks like OCR.

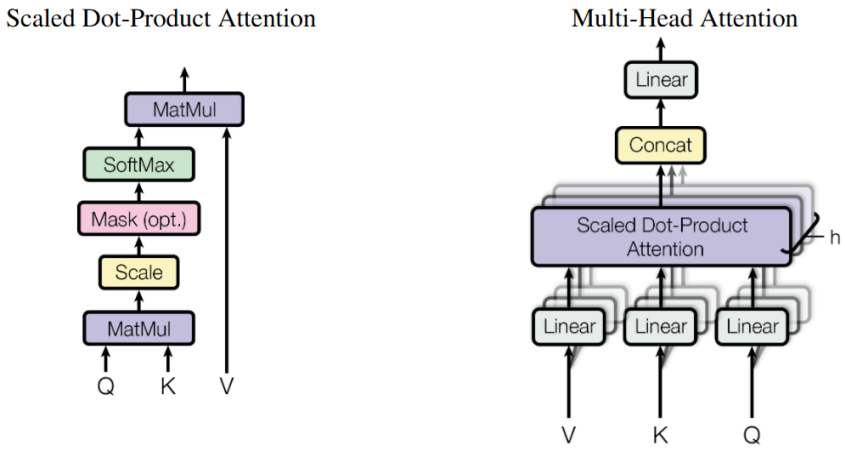


Figure 3.4: Multi-head Attention [4]

The Transformer architecture consists of two main components: the encoder and the decoder. The encoder receives the input sequence and transforms it into a set of rich feature representations by capturing relationships between different input elements using self-attention. This allows the model to understand both local and global contextual information.

The decoder uses the encoded features to generate the output sequence one element at a time. It employs self-attention to understand previously generated outputs and cross-attention to focus on relevant information from the encoder. Together, the encoder and decoder enable the Transformer to effectively model complex input-output relationships, making it suitable for tasks such as text recognition, and OCR.

3.2.3 Vision Transformer (ViT)

ViT is a Transformer-based architecture designed for image recognition tasks [43]. Instead of processing an image as a whole, ViT divides the image into fixed-size patches and treats each patch as a token, similar to words in a sentence. These patch embeddings are then passed through a standard Transformer encoder to learn meaningful visual representations.

Unlike CNNs, which primarily focus on local image features, Vision Transformers can capture both local and global relationships through the self-attention mechanism. This enables the model to understand contextual information across the entire image and learn complex visual patterns more effectively.

Due to its strong feature learning capability, ViT has achieved excellent performance in image classification and has been widely adopted in various computer vision applications [44] [45],

including object detection, image segmentation, scene text recognition, and optical character recognition (OCR).

3.2.4 Tokenizer

Tokenizers convert raw texts into a sequence of smaller units, called tokens, that a model can process numerically. Instead of feeding entire texts directly, they are broken into pieces and mapped to integer IDs from a fixed vocabulary. This is important because neural networks can only work with numbers and the choice of tokenization directly affects how well the model learns.

Bangla words can be broken down into a couple of ways (see Fig. 3.5). The concept of Grapheme was discussed in 1.1, where instead of segmenting the word into individual characters, each word was shown to be formed by combination of graphemes.

The other type of tokenizer explored was Byte Pair Encoding (BPE). It starts by treating every character as a separate token, then repeatedly finds the most frequently occurring pair of neighboring characters or subwords and merges them into a single token. Over time, common character combinations form larger units that often correspond to meaningful parts of words or even complete words. This helps reduce the vocabulary size while still allowing the model to represent unseen words by breaking them into familiar subword units [46] [47].

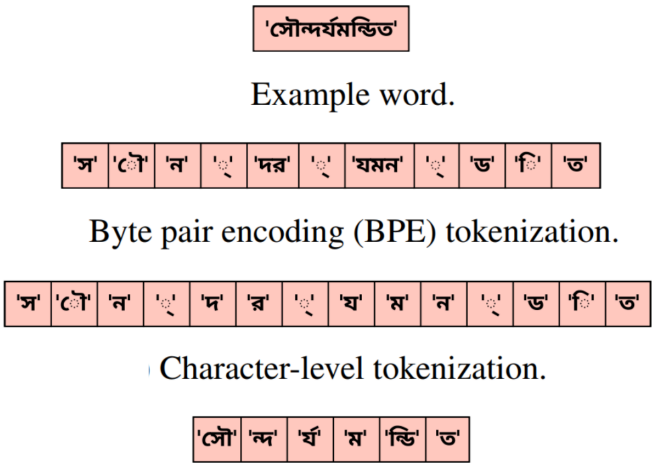


Figure 3.5: Different types of tokenizers [5]

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Experimental Setup

4.1.1 Dataset and Splits

All primary experiments use the BnHTRD dataset [1], a large-scale benchmark of handwritten Bangla word images sourced from real-world documents (Fig. 1.1b shows the sample word images). An `is_bangla_word` filter retains only samples whose labels consist of more than 80 % characters from the Bangla Unicode block (`U+0980`–`U+09FF`), yielding approximately 66,133 usable samples. The data are partitioned into training, validation, and test sets in an 80/10/10 ratio using a fixed random seed of 42, producing 52,905 training, 6,614 validation, and 6,614 test samples. This protocol is held constant across all five experiments to ensure fair comparison.

To assess cross-dataset generalisation, the best-performing model is additionally evaluated on the IIIT-Indic-HW-Words Bengali subset [48], a publicly available handwritten word dataset collected independently of BnHTRD. No fine-tuning or adaptation is performed; the model is applied directly to the IIIT-Indic test split of 17,574 samples using the checkpoint trained on BnHTRD.

For scene text recognition, we use the Bengali word crop subset of the IndicSTR12 dataset [2] (Fig. 1.1c shows the sample images from the dataset), yielding 3,068 real labelled samples split 80/10/10 into training, validation, and test sets. This experiment is described in detail in Section 4.2.2.

4.1.2 Model Variants

Table 4.1 summarises all the configurations that were evaluated. Model names follow the format `[Encoder]_[Tokeniser]`, where the encoder is either a ResNet-18 CNN [3] or the pretrained TrOCR Vision Transformer [49], and the tokeniser is one of grapheme-cluster, character-level, or SentencePiece BPE [47].

Table 4.1: Model configurations evaluated in this work. † ResNet-18 + Character uses an unfiltered dataset (71,099 samples) with a 70/15/15 split and is included for reference only; results are not directly comparable.

Model	**Tokeniser**	**Encoder**	**Params**
ResNet_Grapheme	Grapheme	ResNet-18	37.4 M
ResNet_Char†	Character	ResNet-18	37.5 M
TrOCR_Grapheme	Grapheme	TrOCR ViT	∼334 M
TrOCR_Char	Character	TrOCR ViT	∼334 M
TrOCR_BPE	BPE	TrOCR ViT	∼285 M

4.1.3 Evaluation Metrics

We report two primary metrics: Character Error Rate (CER) and Word Error Rate (WER). CER measures the fraction of characters that are wrong, averaged over the entire test set:

$$\text{CER} = \frac{\text{total character edits needed across all predictions}}{\text{total characters in all ground-truth labels}}, \tag{4.1}$$

where "character edits" counts the minimum number of insertions, deletions, and substitutions required to turn each predicted string into the correct one (Levenshtein distance). A CER of 0 means every character was correct; a CER of 1 means the output was entirely wrong on average.

WER is the fraction of words for which the predicted string does not exactly match the ground truth:

$$\text{WER} = \frac{\text{number of incorrectly predicted words}}{\text{total number of words}}. \tag{4.2}$$

Word Accuracy is reported as $1 - \text{WER}$. Edit distance is computed at the Unicode character level for all models, regardless of tokeniser.

4.2 Results

4.2.1 Handwritten Text Recognition

Table 4.2 presents test-set results for all five configurations on BnHTRD. **TrOCR_Grapheme achieves the best performance on every metric**, attaining a Word Accuracy of 92.21 % and a Corpus CER of 2.52 %.

Table 4.3 presents the cross-dataset evaluation of TrOCR_Grapheme on the IIIT-Indic-HW-Words Bengali test set [48], applied zero-shot without any adaptation. Table 4.4 compares our model with existing approaches on the BnHTRD [1] dataset. It shows that our model outperforms the other models across all metrics.

Table 4.2: Recognition performance on the BnHTRD test set (6,614 samples). The best result in each column is **bold**. † Evaluated on a different test split (10,665 samples) and is not directly comparable.

Model	Tokeniser	Word Acc. (%) ↑	CER (%) ↓	WER (%) ↓	CER = 0 (%) ↑	CER > 0.5 (%) ↓
ResNet_Grapheme	Grapheme	83.88	8.36	16.12	83.9	4.9
ResNet_Char†	Character	60.01	12.66	39.99	60.0	7.0
TrOCR_Grapheme	Grapheme	**92.21**	**2.52**	**7.79**	**92.2**	**0.9**
TrOCR_Char	Character	88.54	3.30	11.46	88.5	1.0
TrOCR_BPE	BPE	87.77	4.25	12.23	87.8	1.9

Table 4.3: Cross-dataset evaluation of TrOCR_Grapheme. The model is trained exclusively on BnHTRD and evaluated on both datasets without re-training or fine-tuning.

Evaluation Dataset	Samples	Word Acc. (%) ↑	CER (%) ↓	WER (%) ↓	CER = 0 (%) ↑	CER > 0.5 (%) ↓
BnHTRD (in-domain)	6,614	92.21	2.52	7.79	92.2	0.9
IIIT-Indic Bengali (out-of-domain)	17,574	42.59	20.66	57.41	42.6	9.8

Table 4.4: Comparison of selected models evaluated on the BnHTRD dataset. Dashes indicate metrics not reported in the respective work. † Evaluated on a different BnHTRD split (same-domain). ‡ GraDeT-HTR evaluates on the full BnHTRD word set. § Evaluated on a filtered subset of 66,133 samples (80/10/10 split). ¶E valuated on a 500-sample validation draw.

Model	Source	Word Acc. (%) ↑	WER (%) ↓	CER (%) ↓
CNN†	[50]	76.40	23.60	—
CNN-Recurrent-CTC†	[51]	82.60	17.40	—
GraDet HTR‡	[5]	—	14.20	6.19
Qwen2-VL-2B (LoRA)¶	This work	88.60	11.40	5.34
TrOCR_Grapheme§	This work	**92.21**	**7.79**	**2.52**

4.2.2 Scene Text Recognition

Scene text recognition differs fundamentally from handwritten document recognition: words appear on signboards, storefronts, banners, and posters under variable illumination, perspective distortion, complex backgrounds, and mixed font styles. Because the available real Bangla scene text corpus (the Bengali split of [2]) contains only 3,068 labelled word crops after filtering, we supplement it with 20,000 calibrated synthetic scene text images generated using 48 validated Bangla Unicode fonts. The synthetic generator models the real dataset's colour palette, contrast distribution, font size range, skew distribution, blur rate, noise level, and JPEG degradation profile, all derived from K-means clustering and pixel-level statistics on the real images. The final real split comprises 1,534 training, 460 validation, and 1,074 test samples.

Training follows a three-phase curriculum initialised from the TrOCR_Grapheme handwritten checkpoint:

1. **Phase 1 — Synthetic pretraining.** The model is trained on 19,000 synthetic images for 7 epochs to adapt its visual representations to scene-text statistics.
2. **Phase 2 — Real fine-tuning.** The model is fine-tuned on 1,534 real scene text words for 7 epochs at a lower learning rate.
3. **Phase 3 — Mixed fine-tuning.** A combined set of 19,000 synthetic and 9,204 real samples (real data oversampled $6\times$) is used for 5 epochs at a very small learning rate to consolidate gains without catastrophic forgetting.

Then the model was tested on real dataset only.

Table 4.5 reports test-set results at the end of each phase.

Table 4.5: Scene text recognition results on the Bangla scene text test set (1,074 samples) after each training phase. The model is initialised from the TrOCR_Grapheme handwritten checkpoint. The best result in each column is **bold**.

Training Phase	**Word Acc. (%)** $\uparrow$	**CER (%)** $\downarrow$	**WER (%)** $\downarrow$
Phase 1 — Synthetic pretraining	67.13	14.29	32.87
Phase 2 — Real fine-tuning	71.69	12.25	28.31
Phase 3 — Mixed fine-tuning	**72.25**	**11.61**	**27.75**

4.3 Discussion

4.3.1 Effect of the Encoder

Holding the tokeniser constant at grapheme level, replacing the ResNet-18 encoder with the pretrained TrOCR ViT reduces Corpus CER from 8.36 % to 2.52 % and raises Word Accuracy by 8.33 percentage points (Table 4.6). This improvement has two compounding sources. First, the TrOCR backbone was pretrained on large-scale handwritten OCR corpora [49], giving it visual representations already tailored to cursive and conjunct letterforms before any Bangla fine-tuning. Second, the ViT produces a rich spatial sequence of 197 patch embeddings that the cross-attention decoder can attend over selectively, whereas the ResNet-18 produces a coarser feature map from a lower-resolution greyscale input.

Table 4.6: Encoder ablation with grapheme tokenisation held constant.

Encoder	**Params**	**Word Acc. (%)**	**CER (%)**	**Throughput (samples/s)**
ResNet-18 CNN	37.4 M	83.88	8.36	≈ 27
TrOCR ViT	$\sim$334 M	**92.21**	**2.52**	≈ 3.5

The accuracy advantage comes at a latency cost: TrOCR decodes approximately 3.5 samples per second compared with 27 for ResNet-18, making the CNN-based system nearly eight times faster. For latency-sensitive deployments, ResNet_Grapheme therefore remains a practical alternative, offering 83.88 % Word Accuracy at substantially lower computational cost.

4.3.2 Effect of the Tokeniser

The three TrOCR variants provide the cleanest tokeniser ablation, as they share the same encoder, pretrained decoder, and evaluation protocol. The ranking is Grapheme $>$ Character $>$ BPE (Table 4.7), consistent with prior findings that linguistically motivated tokenisation outperforms data-driven subword segmentation for Bangla HTR [52] [5].

Table 4.7: Tokeniser ablation with TrOCR ViT encoder held constant.

Tokeniser	**Word Acc. (%)**	**CER (%)**	**WER (%)**
Grapheme	**92.21**	**2.52**	**7.79**
Character	88.54	3.30	11.46
BPE	87.77	4.25	12.23

Grapheme tokenisation. By encoding each cluster (Grapheme) as a single token, the grapheme tokeniser aligns each decoder step with one visually atomic unit, reduces average sequence

length, and limits error accumulation during autoregressive decoding. Bengali contains approximately 13,000 distinct grapheme variations [10], compared with around 250 in English; conjunct characters also often exhibit visual representations that differ substantially from the individual characters they comprise [52], making this alignment particularly important for the script.

Character tokenisation. Character-level encoding splits conjuncts into their constituent codepoints, requiring the decoder to learn implicitly which codepoints co-occur as visual units. Despite this added complexity, character tokenisation outperforms BPE by 0.77 percentage points in Word Accuracy and 0.95 points in CER. Each codepoint retains a consistent visual identity, a given vowel sign is always a vowel sign, making it easier to localise in the encoder feature map than a BPE token whose boundaries are determined by corpus statistics rather than script structure.

BPE tokenisation. BPE merges high-frequency codepoint pairs greedily, producing tokens that may span partial graphemes with no clear visual boundary. This misalignment is detrimental: even slight handwriting variation in a single character can alter the identity of a merged token, whereas grapheme and character token boundaries are invariant to writing style [5]. Notably, the BPE vocabulary of 2,000 tokens is far larger than the 89 or 964 tokens used by the other strategies, yet delivers the worst accuracy of the three, demonstrating that vocabulary size alone does not explain the performance ordering.

4.3.3 Cross-Dataset Generalisation

To probe how well the best in-domain model transfers to unseen data, we evaluate TrOCR_Grapheme on the IIIT-Indic-HW-Words Bengali test set [48] without additional fine-tuning (Table 4.3). Word Accuracy drops sharply from 92.21 % on BnHTRD to 42.59 % on IIIT-Indic, and Corpus CER rises from 2.52 % to 20.66 %.

This degradation is attributable to three distributional differences between the two datasets. First, IIIT-Indic was collected from writers and document sources independent of BnHTRD, introducing handwriting styles, pen types, and background textures not represented in training. Second, the IIIT-Indic vocabulary spans 12,812 unique words, many of which, particularly rare proper nouns, place names, and domain-specific terms, are absent from the BnHTRD label set, making them inherently harder to decode for a model with no prior exposure. Third, the image acquisition pipeline differs between the two corpora, resulting in different noise profiles and contrast characteristics. Qualitative error analysis confirms that the model handles common, short words reliably (42.6 % of predictions are letter-perfect), but struggles on longer and rarer word forms, where early decoding errors compound through the autoregressive process.

Despite the performance gap, two positive observations emerge. The fraction of catastrophic failures (CER > 0.5) rises only to 9.8 %, compared with 0.9 % in-domain, indicating that the model rarely produces completely unrelated output on out-of-distribution samples. Moreover, a near-perfect rate (CER ≤ 0.1) of 45.4 % suggests that useful character-level knowledge is retained across datasets. Target-domain fine-tuning on even a small portion of IIIT-Indic samples would be expected to close much of this gap, as the grapheme vocabulary covers all Bangla script characters shared between the two corpora.

4.3.4 Error Analysis

CER distribution. Across all models, the CER distribution is strongly bimodal: most predictions are either perfect (CER $= 0$) or substantially wrong (CER > 0.5). The fraction of catastrophic failures decreases monotonically with overall accuracy, from 4.9 % for ResNet_Grapheme down to 0.9 % for TrOCR_Grapheme, indicating that stronger models primarily eliminate complete recognition failures rather than incrementally reducing minor substitutions.

Word length. Error analysis by grapheme count reveals a consistent pattern: accuracy degrades as word length increases, consistent with autoregressive error accumulation over longer sequences. TrOCR_Grapheme degrades most gracefully with length, likely because cross-attention allows the decoder to revisit encoder features at each step.

Common confusions. The most frequent substitutions involve visually similar conjunct consonants and diacritics, particularly the short versus long *i*-matra (hrasva-i and dirgha-i) and conjuncts differing by a single stroke. These errors persist even in the best-performing model, indicating that fine-grained diacritic discrimination remains an open challenge. Under character-level tokenisation, the hasanta (virama, `U+09CD`) and individual vowel signs are the most frequently misidentified codepoints, supporting the hypothesis that conjunct resolution is harder when codepoints are treated individually rather than as pre-grouped visual units [52].

4.3.5 Scene Text Recognition

The three-phase curriculum produces steady gains at each stage (Table 4.5). Phase 1 synthetic pretraining alone achieves 67.13 % Word Accuracy, demonstrating that the calibrated generator produces images close enough to real scene text for useful feature transfer. Phase 2 real fine-tuning raises this to 71.69 %, a gain of 4.56 percentage points from adapting to actual scene text variability. Phase 3 mixed fine-tuning yields a further modest improvement to 72.25 %, with CER falling from 12.25 % to 11.61 %.

The overall scene text accuracy (72.25 %) is substantially lower than the handwritten document

accuracy (92.21 %), which is expected given the domain gap: scene text presents challenges absent from document HTR, including arbitrary backgrounds, perspective distortion, partial occlusion, and inconsistent lighting. The small size of the real training set (1,534 samples) limits how far fine-tuning can close this gap. Qualitative inspection reveals that the most common failure modes are short single-character words misread under heavy background clutter, and multi-word fragments in a single crop where the model attends to only one word. The steady improvement across phases confirms that the synthetic-to-real curriculum is an effective strategy when real annotated scene text is scarce.

CHAPTER 5: EVALUATION

5.1 Assessment of Issues

5.1.1 Societal Issues

The proposed Bangla OCR system has significant societal value by facilitating the digitization and preservation of handwritten and printed Bangla documents. Large collections of educational materials, historical manuscripts, government records, and cultural archives remain inaccessible in digital form. By automating text extraction from handwritten and scene text images, the proposed system can improve information accessibility, support digital archiving efforts, and contribute to the preservation of Bangla linguistic heritage. Furthermore, OCR technology can assist visually impaired individuals through text-to-speech applications and improve access to information in low-resource language settings.

5.1.2 Health and Safety Issues

The developed OCR system does not directly pose significant health or safety risks, as it primarily operates on digital images and textual data. However, recognition errors may affect applications where accurate information extraction is critical, such as medical prescriptions, legal documents, or official records. Therefore, OCR-generated outputs should be verified by human operators before being used in safety-critical or high-stakes decision-making processes.

5.1.3 Legal and Cultural Issues

The use of OCR technology must comply with data privacy, intellectual property, and copyright regulations. Digitization of documents should be performed only with appropriate authorization from document owners or custodians. Additionally, since the proposed system processes Bangla text, it contributes positively to the preservation and promotion of Bangla language and culture. Care must be taken to ensure that cultural and historical documents are digitized accurately to avoid unintended alteration or misrepresentation of their original content.

5.2 Evaluation of Environment and Sustainability

The proposed OCR system contributes to sustainability by reducing dependence on physical document storage and enabling efficient digital archiving. Digital preservation of documents

decreases the need for paper reproduction, printing, and physical storage infrastructure, thereby reducing resource consumption over time. Furthermore, the use of automated OCR systems minimizes manual data-entry efforts, improving operational efficiency in educational institutions, libraries, government organizations, and archival centers.

From an environmental perspective, the training of deep learning models requires computational resources and electrical energy. However, the long-term benefits of large-scale document digitization, preservation of historical records, and reduction of paper-based workflows can outweigh these computational costs. Therefore, the proposed OCR framework supports sustainable information management and contributes to the broader digital transformation of society.

5.3 Ethical Issues

This research was conducted following accepted principles of academic and professional ethics. All datasets used in this study were obtained from publicly available sources or used in accordance with their respective usage policies. Proper attribution has been provided to all referenced works, datasets, models, and research contributions to avoid plagiarism and ensure academic integrity.

In addition, the proposed OCR system was developed and evaluated objectively using established performance metrics without manipulation of experimental results. The limitations of the system have been reported transparently, and no claims have been made beyond the observed experimental findings. The research aims to contribute positively to the development of Bangla language technologies while promoting responsible and ethical use of artificial intelligence.

5.4 Future Works

Although the proposed OCR framework demonstrates strong performance for both Bangla handwritten and scene text recognition, several opportunities remain for future improvement and extension:

- **Development of a Unified Detection and Recognition Pipeline**

  The current research focuses on recognizing text from pre-cropped word images, where the text region is already known. In practical applications, however, text often appears within larger images containing multiple words, lines, or background objects. Future work can extend the proposed framework by integrating a text detection module capable of automatically locating Bangla text regions before recognition. Such an end-to-end OCR system would be more suitable for real-world applications, including document digitization, signboard reading, and automatic text extraction from natural scene images.

- **Collection of Larger Scene Text Datasets**

  The availability of large-scale annotated Bangla scene text datasets remains limited compared to handwritten text datasets. This restricts the ability of deep learning models to learn diverse visual patterns and generalize effectively to real-world environments. Future efforts can focus on collecting and publicly releasing larger and more diverse Bangla scene text datasets containing a wide range of fonts, backgrounds, lighting conditions, viewing angles, image qualities, and environmental variations. Such resources would not only improve model performance but would also facilitate further research in Bangla scene text recognition.

- **Real-Time OCR Applications**

  Although the proposed model achieves strong recognition performance, its computational requirements may limit deployment in real-time environments. Future research can explore model optimization techniques such as knowledge distillation, quantization, and lightweight architectures to reduce inference time and memory consumption. This would enable deployment on mobile devices, embedded systems, and edge platforms, supporting applications such as real-time translation, intelligent document scanning, assistive technologies for visually impaired individuals, and automatic signboard or street-text reading systems.

- **Multilingual OCR Framework**

  In many real-world scenarios, Bangla text frequently appears alongside English and other regional languages. Examples include street signs, advertisements, official documents, educational materials, and digital content. Future work can extend the proposed framework to support multilingual text recognition, allowing a single model to accurately recognize and process text from multiple languages simultaneously. Such a system would be more practical for real-world deployment and could significantly enhance accessibility, information retrieval, and document digitization across multilingual environments.

REFERENCES

[1] Rahman, M.A., Tabassum, N., Paul, M., Pal, R. and Islam, M.K. "Bn-htrd: A benchmark dataset for document level offline bangla handwritten text recognition (htr) and line segmentation." In "Computer Vision and Image Analysis for Industry 4.0," pp. 1–16. Chapman and Hall/CRC, 2023

[2] Lunia, H., Mondal, A. and Jawahar, C.V. "IndicSTR12: A dataset for indic scene text recognition." In "Document Analysis and Recognition – ICDAR 2023 Workshops," pp. 233–250, 2023

[3] Ramzan, F., Khan, M.U.G., Rehmat, A., Iqbal, S., Saba, T., Rehman, A. and Mehmood, Z. "A deep learning approach for automated diagnosis and multi-class classification of alzheimer's disease stages using resting-state fmri and residual neural networks." *Journal of medical systems*, volume 44, no. 2:p. 37, 2020

[4] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I. "Attention is all you need." *Advances in neural information processing systems*, volume 30, 2017

[5] Hasan, M.M., Choudhury, A.N.T., Hasan, M. and Khan, M.M. "Gradet-htr: A resource-efficient bengali handwritten text recognition system utilizing grapheme-based tokenizer and decoder-only transformer." In "Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations," pp. 696–706, 2025

[6] Kaur, A. and Lehal, G.S. "The Evolution and Impact of Optical Character Recognition (OCR) in Digital Transformation." *International Journal of Emerging Trends in Engineering and Development (IJETED)*, volume 15, no. 4:pp. 27–40, 2025. Open Access

[7] Wang, J. "A Study of The OCR Development History and Directions of Development." *Highlights in Science, Engineering and Technology*, volume 72:pp. 409–415, 2023. Open Access

[8] Wang, H., Pan, C., Guo, X., Ji, C. and Deng, K. "From Object Detection to Text Detection and Recognition: A Brief Evolution History of OCR." *WIREs Computational Statistics*, volume 13, no. 5:p. e1547, 2021

[9] Mishra, A., Sikdar, J. and Kumar, S.U. "Deep learning-based optical character recognition for robust real-world conditions: A comparative analysis." In "2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)," pp. 1–7, June 2024

[10] Alam, S., Reasat, T., Sushmit, A.S., Siddique, S.M., Rahman, F., Hasan, M. and Humayun, A.I. "A large multi-target dataset of common bengali handwritten graphemes." In "Document Analysis and Recognition – ICDAR 2021," pp. 383–398, 2021

[11] Basu, S., Das, N., Sarkar, R., Kundu, M., Nasipuri, M. and Basu, D.K. "A hierarchical approach to recognition of handwritten bangla characters." *Pattern Recognition*, volume 42, no. 7:pp. 1467–1484, 2009

[12] Das, N., Das, B., Sarkar, R., Basu, S., Kundu, M. and Nasipuri, M. "Handwritten bangla basic and compound character recognition using mlp and svm classifier." *Journal of Computing*, volume 2, 02 2010

[13] Roy, P.P., Dey, P., Roy, S., Pal, U. and Kimura, F. "A novel approach of bangla handwritten text recognition using hmm." In "2014 14th International Conference on Frontiers in Handwriting Recognition," pp. 661–666, 2014

[14] Roy, P.P., Bhunia, A.K., Das, A., Dey, P. and Pal, U. "Hmm-based indic handwritten word recognition using zone segmentation." *Pattern Recognition*, volume 60:pp. 1057–1075, 2016

[15] Rabby, A.S.A., Haque, S., Islam, S., Abujar, S. and Hossain, S.A. "Bornonet: Bangla handwritten characters recognition using convolutional neural network." *Procedia Computer Science*, volume 143:pp. 528–535, 2018. 8th International Conference on Advances in Computing & Communications (ICACC-2018)

[16] Sarkar, R., Das, N., Basu, S., Kundu, M., Nasipuri, M. and Basu, D.K. "Cmaterdb1: a database of unconstrained handwritten bangla and bangla–english mixed script document image." *International Journal on Document Analysis and Recognition (IJDAR)*, volume 15, no. 1:pp. 71–83, 2012

[17] Bhattacharya, U. and Chaudhuri, B.B. "Handwritten numeral databases of indian scripts and multistage recognition of mixed numerals." *IEEE transactions on pattern analysis and machine intelligence*, volume 31, no. 3:pp. 444–457, 2008

[18] Biswas, M., Islam, R., Shom, G.K., Shopon, M., Mohammed, N., Momen, S. and Abedin, A. "Banglalekha-isolated: A multi-purpose comprehensive dataset of handwritten bangla isolated characters." *Data in Brief*, volume 12:pp. 103–107, 2017

[19] Rabby, A.S.A., Haque, S., Abujar, S. and Hossain, S.A. "Ekushnet: Using convolutional neural network for bangla handwritten recognition." *Procedia Computer Science*, volume 143:pp. 603–610, 2018. 8th International Conference on Advances in Computing & Communications (ICACC-2018)

[20] Chakraborty, P., Islam, A., Abu Yousuf, M., Agarwal, R. and Choudhury, T. "Bangla handwritten character recognition using convolutional neural network." In "Machine Intelligence and Data Science Applications: Proceedings of MIDAS 2021," pp. 721–731. Springer, 2022

[21] Saha, C. and Rahman, M.M. "Banglanet: Bangla handwritten character recognition using ensembling of convolutional neural network.", 2024

[22] Khan, M.M., Uddin, M.S., Parvez, M.Z. and Nahar, L. "A squeeze and excitation resnext-based deep learning model for bangla handwritten compound character recognition." *Journal of King Saud University - Computer and Information Sciences*, volume 34, no. 6, Part B:pp. 3356–3364, 2022

[23] Maung, A.T., Salekin, S. and Haque, M.A. "A hybrid approach to bangla handwritten ocr: combining yolo and an advanced cnn." *Discover Artificial Intelligence*, volume 5, no. 1:p. 119, 2025

[24] Adak, C., Chaudhuri, B.B. and Blumenstein, M. "Offline cursive bengali word recognition using cnns with a recurrent model." In "2016 15th International Conference on Frontiers in Handwriting Recognition (ICFHR)," pp. 429–434, 2016

[25] Safir, F.B., Ohi, A.Q., Mridha, M.F., Monowar, M.M. and Hamid, M.A. "End-to-end optical character recognition for bengali handwritten words." *arXiv preprint arXiv:2105.04020*, 2021

[26] Hossain, M.I., Rakib, M., Mollah, S., Rahman, F. and Mohammed, N. "Lila-boti: Leveraging isolated letter accumulations by ordering teacher insights for bangla handwriting recognition." *arXiv preprint arXiv:2205.11420*, 2022

[27] Ahmed, T.S.B., Rahman, T., Biswas, S., Sabuj, S.R., Bhuian, M.B., Moni, M.A. and Alam, M.A. "A vision transformer-based hybrid neural architecture for automated handwritten bangla character recognition and braille conversion." *Knowledge-Based Systems*, p. 114546, 2025

[28] Islam, S.J., Hasan, M.M., Ovi, M.H., Rabby, A.S.A. and Rahman, F. "Bornodrishti: Leveraging vision encoders and domain-adaptive learning for bangla ocr on diverse documents." In "Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 5: Industry Track)," pp. 278–286, 2026

[29] Gupta, M.K., Dhawan, S. and Kumar, A. "Cmvit ocr: printed indian language recognition using cmvit." In "2024 11th International Conference on Signal Processing and Integrated Networks (SPIN)," pp. 28–33. IEEE, 2024

[30] Islam, R., Islam, M.R. and Talukder, K.H. "Extraction and recognition of bangla texts from natural scene images using cnn." In "International Conference on Image and Signal Processing," pp. 243–253. Springer, 2020

[31] Islam, R., Islam, M.R. and Talukder, K.H. "An efficient roi detection algorithm for bangla text extraction and recognition from natural scene images." *Journal of King Saud University-Computer and Information Sciences*, volume 34, no. 8:pp. 6150–6164, 2022

[32] Dutta, M., Das, U.K., Datta, N. and Das, K.J. "Multilingual scene text recognition: A faster r-cnn approach for bengali and english scripts." *Expert Systems with Applications*, p. 130482, 2025

[33] Mazumder, T., Nusrat, F., Mahi, A.B.S., Brishty, J.B., Rahman, R. and Helaly, T. "Automated and efficient bangla signboard detection, text extraction, and novel categorization method for underrepresented languages in smart cities." *Results in Engineering*, volume 26:p. 105156, 2025

[34] Das, N. et al. "Handwritten Bengali word image dataset.", 2022. 113,076 word images with ground truth transcriptions, available via Mendeley Data

[35] Mridha, M.F., Ohi, A.Q., Ali, M.M., Emon, M.R.I. and Kabir, M. "BanglaWriting: A multipurpose offline Bangla handwriting dataset." *Data in Brief*, volume 34:p. 106633, 2021

[36] Nayef, N., Yin, F., Bizid, I., Choi, H.I., Feng, Y., Karatzas, D., Luo, Z., Pal, U., Rigaud, C., Chazalon, J. et al. "ICDAR2017 robust reading challenge on multi-lingual scene text detection and script identification — RRC-MLT." In "14th IAPR International Conference on Document Analysis and Recognition (ICDAR)," volume 1, pp. 1454–1459. IEEE, 2017

[37] Nayef, N., Patel, Y., Busta, M., Chowdhury, P.N., Karatzas, D., Khlif, W., Matas, J., Pal, U., Burie, J.C., lin Liu, C. et al. "ICDAR2019 robust reading challenge on multi-lingual scene text detection and recognition — RRC-MLT-2019." In "2019 International Conference on Document Analysis and Recognition (ICDAR)," pp. 1582–1587. IEEE, 2019

[38] Li, Y., Chen, D., Tang, T. and Shen, X. "HTR-VT: Handwritten text recognition with vision transformer." *Pattern Recognition*, volume 158:p. 110967, 2025

[39] Alhussein,. "A two-stage road sign detection and text recognition system based on YOLOv7." *Internet of Things*, volume 27:p. 101330, 2024

[40] O'shea, K. and Nash, R. "An introduction to convolutional neural networks." *arXiv preprint arXiv:1511.08458*, 2015. ArXiv

[41] He, K., Zhang, X., Ren, S. and Sun, J. "Deep residual learning for image recognition." In "Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)," , June 2016

[42] Glorot, X. and Bengio, Y. "Understanding the difficulty of training deep feedforward neural networks." In "Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics," volume 9, pp. 249–256, 2010

[43] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J. and Houlsby, N. "An image is worth 16x16 words: Transformers for image recognition at scale." In "International Conference on Learning Representations," , 2021

[44] Huo, Y., Jin, K., Cai, J., Xiong, H. and Pang, J. "Vision transformer (vit)-based applications in image classification." In "2023 IEEE 9th Intl Conference on Big Data Security on Cloud (BigDataSecurity), IEEE Intl Conference on High Performance and Smart Computing, (HPSC) and IEEE Intl Conference on Intelligent Data and Security (IDS)," pp. 135–140, 2023

[45] Al-Hammuri, K., Gebali, F., Kanan, A. and Chelvan, I.T. "Vision transformer architecture and applications in digital health: a tutorial and survey." *Visual computing for industry, biomedicine, and art*, volume 6, no. 1:p. 14, 2023. Open Access

[46] Gage, P. "A new algorithm for data compression." *C Users J.*, volume 12, no. 2:p. 23–38, February 1994

[47] Sennrich, R., Haddow, B. and Birch, A. "Neural machine translation of rare words with subword units." In "Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)," pp. 1715–1725, 2016

[48] Gongidi, S. and Jawahar, C.V. "IIIT-Indic-HW-Words: A dataset for indic handwritten text recognition." In "International Conference on Document Analysis and Recognition," pp. 444–459, 2021

[49] Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z. and Wei, F. "Trocr: Transformer-based optical character recognition with pre-trained models." In "Proceedings of the AAAI conference on artificial intelligence," volume 37, pp. 13094–13102, 2023

[50] Tusher, M.M.R., Farid, F.A., Al-Hasan, M., Miah, A.S.M., Rinky, S.R., Jim, M.H. et al. "Development of a lightweight model for handwritten dataset recognition: Bangladeshi city names in Bangla script." *Computers, Materials & Continua*, volume 80, no. 2:pp. 2633–2656, 2024

[51] Adak, C., Chaudhuri, B.B. and Blumenstein, M. “Offline cursive Bengali word recognition using CNNs with a recurrent model.” In “Proceedings of the 15th International Conference on Frontiers in Handwriting Recognition (ICFHR),” pp. 429–434, 2016

[52] Basher, M.J.I., Noor, M.R., Afroze, S., Ahmed, I. and Hoque, M.M. “Bngraphemizer: A grapheme-based tokenizer for bengali handwritten text recognition.” In “2023 IEEE 9th International Women in Engineering (WIE) Conference on Electrical and Computer Engineering (WIECON-ECE),” pp. 183–188. IEEE, 2023

CHAPTER A: CODES

A.1 Dataset loading, model training and testing

```
import os, glob, json, random, math, warnings, time
from collections import Counter
import numpy as np
import pandas as pd
import cv2
from PIL import Image, ImageDraw, ImageFont
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from sklearn.manifold import TSNE
import editdistance
from tqdm import tqdm


import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from transformers import (
    VisionEncoderDecoderModel,
    AutoImageProcessor,
    Seq2SeqTrainer,
    Seq2SeqTrainingArguments,
    EarlyStoppingCallback,
    TrainerCallback,
)
from sklearn.model_selection import train_test_split



# Special token ids
# We build a custom vocabulary from graphemes.
# These 4 ids are reserved and must match what the decoder config expects.
PAD_ID  = 0
UNK_ID  = 1
BOS_ID  = 2
EOS_ID  = 3
SPECIAL_TOKENS = 4      # number of reserved slots before real graphemes start
MAX_LEN = 64            # max decoder sequence length (generous for long words)

# Paths
ROOT        = "/kaggle/input/datasets/kaidul007/bnhtrd/Dataset"
PUNCT       = {'.', ':', '    ', ',', ';', '!', '?', '-', '-', '    '}
CKPT_DIR      = '/kaggle/working/trocr_grapheme'
# Read-only input path that contains checkpoints from the previous Kaggle run.
# epoch_* folders are expected at  PREV_CKPT_DIR/trocr_grapheme/epoch_XX/
PREV_CKPT_DIR = '/kaggle/input/notebooks/irtisam/trocr-vit-grapheme/trocr_grapheme'
RESULTS_CSV = '/kaggle/working/test_results_trocr_grapheme.csv'
PLOT_DIR    = '/kaggle/working/plots_trocr_grapheme'
```

```python


# 1.  SHARED BASELINE CONFIG  (identical across all 5 experiments)

class Config:
    # encoder
    TROCR_MODEL     = 'microsoft/trocr-base-handwritten'

    # training (FIXED - do not change across experiments)
    BATCH_SIZE      = 16
    EVAL_BATCH_SIZE = 16
    EPOCHS          = 5         #     shared baseline
    LR              = 5e-5
    WARMUP_STEPS    = 500
    WEIGHT_DECAY    = 0.01
    TEST_SPLIT      = 0.10        #     shared baseline
    VAL_SPLIT       = 0.10        #     shared baseline
    SEED            = 42          #     shared baseline
    FP16            = True        # set False if CPU-only

random.seed(Config.SEED)
np.random.seed(Config.SEED)
torch.manual_seed(Config.SEED)
os.makedirs(PLOT_DIR,   exist_ok=True)
os.makedirs(CKPT_DIR,   exist_ok=True)

device = (torch.accelerator.current_accelerator().type
          if torch.accelerator.is_available() else 'cpu')
print(f"Using device: {device}")


# 2.  DATA LOADING  (identical across all 5 experiments)
def is_bangla_word(text: str) -> bool:
    """True when >80 % of non-space chars are in the Bangla Unicode block."""
    chars = [c for c in text if not c.isspace()]
    if not chars:
        return False
    bangla = sum(1 for c in chars if '\u0980' <= c <= '\u09FF')
    return bangla / len(chars) > 0.8


def load_dataset(root: str) -> pd.DataFrame:
    label_dfs = []
    for file in glob.glob(root + "/*/*.xlsx"):
        try:
            sheets = pd.read_excel(file, sheet_name=None, engine="openpyxl")
            for df in sheets.values():
                if df.shape[1] < 2:
                    continue
                df = df.iloc[:, :2].copy()
                df.columns = ["Id", "Word"]
                label_dfs.append(df)
        except Exception:
            continue

    labels_df = pd.concat(label_dfs, ignore_index=True)
    images    = glob.glob(root + "/*/Words/*/*.jpg")
    images   += glob.glob(root + "/*/Words/*/*.JPG")

```

```
    image_df = pd.DataFrame({
        "Id":         [os.path.basename(p).split('.')[0] for p in images],
        "image_path": images,
    })

    df = (image_df
          .merge(labels_df, on="Id", how="inner")
          .rename(columns={"Word": "label"})[["image_path", "label"]]
          .dropna()
          .reset_index(drop=True))

    df['label'] = df['label'].astype(str).str.strip()
    df = df[df['label'] != '']
    df = df[~df['label'].isin(PUNCT)]
    df = df[df['label'].apply(is_bangla_word)]   # same filter as TrOCR+BPE run
    df = df[df['image_path'].apply(os.path.exists)].reset_index(drop=True)
    print(f"Total usable samples: {len(df):,}")
    return df

# 3.  BANGLA GRAPHEME SEGMENTER
#     Identical to the ResNet+Grapheme experiments (Doc 2/3).
#     A grapheme = one visual unit: consonant cluster + optional vowel sign.
HASANTA          = '\u09CD'
CHANDRABINDU     = '\u0981'
ANUSVARA         = '\u0982'
VISARGA          = '\u0983'
NUKTA            = '\u09BC'

VOWEL_SIGNS = set(
    '\u09BE\u09BF\u09C0\u09C1\u09C2\u09C3\u09C4'
    '\u09C7\u09C8\u09CB\u09CC\u09D7'
)
INDEPENDENT_VOWELS = set(
    '\u0985\u0986\u0987\u0988\u0989\u098A\u098B'
    '\u098F\u0990\u0993\u0994'
)
CONSONANTS = set(chr(c) for c in range(0x0995, 0x09BA)) | {
    '\u09DC', '\u09DD', '\u09DF', '\u09CE', '\u09B0'
}


def segment_graphemes(word: str) -> list:
    """
    Segment a Bangla word into a list of grapheme strings.
    Each grapheme is one of:
          independent vowel  (+ optional anusvara/visarga/chandrabindu)
          consonant cluster  (C (hasanta C)*)  + optional vowel sign
          any other character treated as a single grapheme
    """
    graphemes = []
    i, n = 0, len(word)
    while i < n:
        ch = word[i]
        if ch in INDEPENDENT_VOWELS:
            g = ch; i += 1
            while i < n and word[i] in {CHANDRABINDU, ANUSVARA, VISARGA}:
                g += word[i]; i += 1
            graphemes.append(g)
        elif ch in CONSONANTS:
```

```
            g = ch; i += 1
            if i < n and word[i] == NUKTA:
                g += word[i]; i += 1
            # absorb hasanta + following consonant  (conjunct consonants)
            while i + 1 < n and word[i] == HASANTA and word[i + 1] in CONSONANTS:
                g += word[i] + word[i + 1]; i += 2
            # absorb trailing vowel sign
            if i < n and word[i] in VOWEL_SIGNS:
                g += word[i]; i += 1
            graphemes.append(g)
        else:
            graphemes.append(ch); i += 1
    return graphemes



# 4.  GRAPHEME VOCABULARY  (this file's key difference vs BPE)

class GraphemeVocab:
    """
    Minimal vocabulary built from the training set's grapheme inventory.
    Maps every unique grapheme     integer id, keeping 4 special slots.

    Compared with SentencePiece BPE:
      + Linguistically motivated     each token = one visual unit
      + Fully interpretable vocabulary
      + No hyper-parameter (vocab size) to tune
      - Larger effective vocab than BPE for the same coverage
      - Longer sequences on average (no subword merging)
    """

    def __init__(self):
        self.g2i: dict = {}
        self.i2g: dict = {
            PAD_ID: '<pad>',
            UNK_ID: '<unk>',
            BOS_ID: '<s>',
            EOS_ID: '</s>',
        }
        self.vocab_size: int = SPECIAL_TOKENS

    # build from training labels
    def build(self, labels) -> 'GraphemeVocab':
        all_g = sorted({
            g
            for word in labels
            for g in segment_graphemes(str(word))
        })
        for idx, g in enumerate(all_g):
            tid = idx + SPECIAL_TOKENS
            self.g2i[g]   = tid
            self.i2g[tid] = g
        self.vocab_size = SPECIAL_TOKENS + len(all_g)
        print(f"Grapheme vocab: {len(all_g)} unique graphemes  "
              f"(vocab_size = {self.vocab_size})")
        return self

    #  serialise / deserialise
    def save(self, path: str):
```

```
        with open(path, 'w', encoding='utf-8') as f:
            json.dump({'g2i': self.g2i,
                       'i2g': {str(k): v for k, v in self.i2g.items()},
                       'vocab_size': self.vocab_size}, f, ensure_ascii=False)

    @classmethod
    def load(cls, path: str) -> 'GraphemeVocab':
        with open(path, encoding='utf-8') as f:
            d = json.load(f)
        v = cls()
        v.g2i        = d['g2i']
        v.i2g        = {int(k): val for k, val in d['i2g'].items()}
        v.vocab_size = d['vocab_size']
        return v

    # encode / decode
    def encode(self, word: str) -> list:
        """
        word     list of int ids, WITHOUT BOS/EOS.
        BOS/EOS are added by encode_label() below.
        """
        return [self.g2i.get(g, UNK_ID) for g in segment_graphemes(str(word))]

    def decode(self, ids) -> str:
        """ids (list/tensor)     string, stopping at EOS, skipping specials."""
        out = []
        for t in ids:
            t = int(t)
            if t == EOS_ID:
                break
            if t in (PAD_ID, BOS_ID, UNK_ID):
                continue
            out.append(self.i2g.get(t, ''))
        return ''.join(out)


def encode_label(vocab: GraphemeVocab, text: str) -> list:
    """
    Encode a word to a padded id sequence:
        [BOS, g1, g2,    , EOS, PAD, PAD,    ]   length = MAX_LEN
    Used to build the labels tensor fed to the decoder.
    """
    ids = vocab.encode(text)
    seq = ([BOS_ID] + ids + [EOS_ID])[:MAX_LEN]
    seq += [PAD_ID] * (MAX_LEN - len(seq))
    return seq


# 5.  DATASET
#
class TrOCRGraphemeDataset(Dataset):
    """
    PyTorch Dataset for TrOCR with grapheme labels.

    Image preprocessing is delegated entirely to HuggingFace's
    AutoImageProcessor (ViTImageProcessor under the hood), which:
          resizes to 384 384
          normalises with ImageNet mean/std
    So we only need to hand it a PIL RGB image.
```

```
287
288     Labels are encoded with GraphemeVocab.
289     Positions that correspond to PAD_ID are set to -100 so
290     CrossEntropyLoss ignores them (standard HF convention).
291     """
292
293     def __init__(self, df: pd.DataFrame,
294                  vocab: GraphemeVocab,
295                  feature_extractor):
296         self.df               = df.reset_index(drop=True)
297         self.vocab            = vocab
298         self.feature_extractor = feature_extractor
299
300     def __len__(self):
301         return len(self.df)
302
303     def __getitem__(self, idx):
304         # load image
305         img_bgr = cv2.imread(self.df.iloc[idx, 0])
306         if img_bgr is None:
307             img_rgb = Image.new('RGB', (384, 384), color=255)
308         else:
309             img_rgb = Image.fromarray(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
310
311         pixel_values = self.feature_extractor(
312             img_rgb, return_tensors='pt'
313         ).pixel_values.squeeze(0)                    # (3, 384, 384)
314
315         #  encode label
316         label_str = str(self.df.iloc[idx, 1])
317         label_ids = encode_label(self.vocab, label_str)
318         labels    = torch.tensor(label_ids, dtype=torch.long)
319         labels[labels == PAD_ID] = -100              # mask padding for loss
320
321         return {'pixel_values': pixel_values, 'labels': labels}
322
323     def get_path(self, idx):  return self.df.iloc[idx, 0]
324     def get_label(self, idx): return str(self.df.iloc[idx, 1])
325
326 # 6.  MODEL SETUP  (encoder identical to TrOCR+BPE; only vocab size differs)
327 def build_model(vocab: GraphemeVocab) -> VisionEncoderDecoderModel:
328     """
329     Load pretrained TrOCR and swap the decoder vocabulary for our
330     grapheme vocabulary.
331
332     The ViT encoder weights are kept exactly as in the BPE experiment.
333     Only the decoder embedding matrix and LM head are re-initialised
334     to the new vocab size     giving us a clean tokenizer-only comparison.
335     """
336     print(f"Loading {Config.TROCR_MODEL}     ")
337     model = VisionEncoderDecoderModel.from_pretrained(Config.TROCR_MODEL)
338
339     new_vocab = vocab.vocab_size
340
341     # resize decoder embedding + LM head
342     model.decoder.resize_token_embeddings(new_vocab)
343
344     # update every config slot that references vocab size
345     model.config.vocab_size             = new_vocab
```

```
    model.config.decoder.vocab_size     = new_vocab
    model.config.decoder_start_token_id = BOS_ID
    model.config.pad_token_id           = PAD_ID
    model.config.eos_token_id           = EOS_ID
    model.config.bos_token_id           = BOS_ID

    # generation config
    model.generation_config.decoder_start_token_id = BOS_ID
    model.generation_config.pad_token_id           = PAD_ID
    model.generation_config.eos_token_id           = EOS_ID
    model.generation_config.bos_token_id           = BOS_ID
    model.generation_config.max_new_tokens          = MAX_LEN

    total     = sum(p.numel() for p in model.parameters())
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    print(f"Parameters: {total/1e6:.1f}M total  |  {trainable/1e6:.1f}M trainable")
    print(f"Decoder vocab size: {new_vocab} graphemes")
    return model


# 7.  METRICS
def cer_single(pred: str, gt: str) -> float:
    if not gt:
        return 0.0 if not pred else 1.0
    return editdistance.eval(list(pred), list(gt)) / len(gt)


def corpus_cer_wer(preds, gts):
    total_dist = total_chars = total_word_err = 0
    for p, g in zip(preds, gts):
        total_dist     += editdistance.eval(list(p), list(g))
        total_chars    += max(len(g), 1)
        total_word_err += (0 if p == g else 1)
    return (total_dist / max(total_chars, 1),
            total_word_err / max(len(gts), 1))


# 8.  COMPUTE_METRICS FOR TRAINER
def make_compute_metrics(vocab: GraphemeVocab):
    """
    Returns a compute_metrics closure for Seq2SeqTrainer.
    Called after each eval epoch with:
      pred.predictions : (N, seq_len)  int array   generated token ids
      pred.label_ids   : (N, seq_len)  int array   ground truth (-100 for pad)
    """
    def compute_metrics(pred):
        pred_ids  = pred.predictions
        label_ids = pred.label_ids.copy()

        # restore masked positions so decode doesn't crash
        label_ids[label_ids == -100] = PAD_ID

        # clip any out-of-range ids from generate()
        pred_ids  = np.clip(pred_ids,  0, vocab.vocab_size - 1)
        label_ids = np.clip(label_ids, 0, vocab.vocab_size - 1)

        preds = [vocab.decode(row) for row in pred_ids]
        gts   = [vocab.decode(row) for row in label_ids]
```

```
        cer, wer = corpus_cer_wer(preds, gts)
        return {'cer': round(cer, 4), 'wer': round(wer, 4)}

    return compute_metrics

# 9.  TRAINING
def train_model(model, train_dataset, val_dataset,
                vocab: GraphemeVocab,
                start_epoch: int = 0) -> 'Seq2SeqTrainer':
    """
    start_epoch : number of epochs already completed before this call.
                  EpochCheckpointCallback adds this offset so the saved
                  folders are epoch_(start_epoch+1), epoch_(start_epoch+2)
                  Pass 0 (default) for a brand-new run.
    """

    args = Seq2SeqTrainingArguments(
        output_dir                  = CKPT_DIR,
        num_train_epochs            = Config.EPOCHS,
        per_device_train_batch_size = Config.BATCH_SIZE,
        per_device_eval_batch_size  = Config.EVAL_BATCH_SIZE,
        learning_rate               = Config.LR,
        warmup_steps                = Config.WARMUP_STEPS,
        weight_decay                = Config.WEIGHT_DECAY,
        predict_with_generate       = True,
        fp16                        = Config.FP16 and (device != 'cpu'),
        eval_strategy               = 'epoch',
        save_strategy               = 'epoch',
        save_total_limit            = 2,
        load_best_model_at_end      = True,
        metric_for_best_model       = 'cer',
        greater_is_better           = False,
        logging_steps               = 100,
        logging_dir                 = f'{CKPT_DIR}/logs',
        report_to                   = 'none',
        dataloader_num_workers      = 2,
        remove_unused_columns       = False,
        seed                        = Config.SEED,
    )

    #  EpochCheckpointCallback: saves model weights keyed by epoch number
    class EpochCheckpointCallback(TrainerCallback):
        """
        Saves model weights after every epoch into a clearly named sub-folder:
            <CKPT_DIR>/epoch_<N>/
        This is separate from the HF Trainer's own save_strategy='epoch'
        checkpoints so that every epoch's weights are always preserved,
        regardless of save_total_limit.
        """
        def on_epoch_end(self, args, state, control, model=None, **kwargs):
            # state.epoch always resets to 1,2,    each trainer.train() call.
            # Adding start_epoch gives the true global epoch number.
            global_epoch = int(round(state.epoch)) + start_epoch
            save_path = os.path.join(CKPT_DIR, f"epoch_{global_epoch:02d}")
            os.makedirs(save_path, exist_ok=True)
            model.save_pretrained(save_path)
            print(f"  [EpochCheckpoint] Saved epoch {global_epoch} weights     {save_path}")

    trainer = Seq2SeqTrainer(
```

```
        model           = model,
        args            = args,
        train_dataset   = train_dataset,
        eval_dataset    = val_dataset,
        compute_metrics = make_compute_metrics(vocab),
        callbacks       = [
            EarlyStoppingCallback(early_stopping_patience=3),
            EpochCheckpointCallback(),
        ],
    )

    print(f"\n{'   '*60}")
    print(f"  TrOCR ViT  +  Grapheme Tokenizer")
    if start_epoch > 0:
        print(f"  Resuming from epoch {start_epoch}       "
              f"saving as epoch_{start_epoch+1:02d}     "
              f"epoch_{start_epoch+Config.EPOCHS:02d}")
    else:
        print(f"  Training for up to {Config.EPOCHS} epochs on {device}")
    print(f"  Grapheme vocab size : {vocab.vocab_size}")
    print(f"{'   '*60}\n")

    trainer.train()
    return trainer

# 9b.  CHECKPOINT LOADER
#      Load a previously saved epoch checkpoint for further training or
#      evaluation.  Works with both the per-epoch folders written by
#      EpochCheckpointCallback and the HF Trainer's own checkpoint folders.
def load_checkpoint(
    epoch: int | None = None,
    checkpoint_dir: str = CKPT_DIR,
    prev_checkpoint_dir: str = PREV_CKPT_DIR,
    vocab: GraphemeVocab | None = None,
) -> tuple[VisionEncoderDecoderModel, int]:
    """
    Load model weights saved by EpochCheckpointCallback.

    Search order (when epoch=None):
      1. checkpoint_dir  (CKPT_DIR      writable working output)
      2. prev_checkpoint_dir  (PREV_CKPT_DIR     read-only Kaggle input)
    The folder with the highest epoch number across both locations wins.

    Parameters
    ----------
    epoch : int or None
        If given, loads from  <checkpoint_dir>/epoch_<N>/  (working dir first,
        then prev_checkpoint_dir as fallback).
        If None, auto-selects the latest epoch across both directories.
    checkpoint_dir : str
        Writable output folder (default: CKPT_DIR).
    prev_checkpoint_dir : str
        Read-only input folder from a previous Kaggle run (default: PREV_CKPT_DIR).
    vocab : GraphemeVocab or None
        When provided, validates vocab size and restores generation config.

    Returns
    -------
    (model, loaded_epoch)     model on `device`, loaded_epoch as int.
```

```
    loaded_epoch is used as start_epoch for the next train_model() call.

    Example     resume from latest checkpoint across both dirs
    ------------------------------------------------------------
    vocab  = GraphemeVocab.load(f'{PREV_CKPT_DIR}/grapheme_vocab.json')
    model, start_epoch = load_checkpoint(vocab=vocab)
    trainer = train_model(model, train_dataset, val_dataset, vocab,
                          start_epoch=start_epoch)
    """
    if epoch is None:
        # Collect epoch_* folders from BOTH dirs, pick the highest epoch number.
        def _collect(d):
            return [f for f in glob.glob(os.path.join(d, "epoch_*"))
                    if os.path.isdir(f)]
        all_folders = _collect(checkpoint_dir) + _collect(prev_checkpoint_dir)
        all_folders = sorted(
            all_folders,
            key=lambda p: int(os.path.basename(p).split("_")[-1]),
        )
        if not all_folders:
            raise FileNotFoundError(
                f"No epoch_* folders found in:\n"
                f"  {checkpoint_dir}\n"
                f"  {prev_checkpoint_dir}"
            )
        ckpt_path = all_folders[-1]
        loaded_epoch = int(os.path.basename(ckpt_path).split("_")[-1])
        print(f"[load_checkpoint] Auto-selected latest checkpoint: "
              f"epoch {loaded_epoch} ({ckpt_path})")
    else:
        # Try writable dir first, then fall back to read-only input dir.
        ckpt_path = os.path.join(checkpoint_dir, f"epoch_{epoch:02d}")
        if not os.path.isdir(ckpt_path):
            ckpt_path = os.path.join(prev_checkpoint_dir, f"epoch_{epoch:02d}")
        if not os.path.isdir(ckpt_path):
            raise FileNotFoundError(
                f"epoch_{epoch:02d} not found in either:\n"
                f"  {checkpoint_dir}\n"
                f"  {prev_checkpoint_dir}"
            )
        loaded_epoch = epoch
        print(f"[load_checkpoint] Loading epoch {loaded_epoch} from {ckpt_path}")

    model = VisionEncoderDecoderModel.from_pretrained(ckpt_path).to(device)

    # Restore generation config so the model works out-of-the-box
    model.generation_config.decoder_start_token_id = BOS_ID
    model.generation_config.pad_token_id           = PAD_ID
    model.generation_config.eos_token_id           = EOS_ID
    model.generation_config.bos_token_id           = BOS_ID
    model.generation_config.max_new_tokens          = MAX_LEN

    if vocab is not None:
        # Sanity check: loaded vocab size must match the model's embedding dim
        expected = vocab.vocab_size
        actual   = model.config.decoder.vocab_size
        if expected != actual:
            print(
                f"       vocab mismatch: vocab says {expected}, "
```

```
                f"model config says {actual}. "
                "Pass the correct GraphemeVocab or re-build the vocabulary."
            )
        else:
            print(f"       Vocab size matches: {actual}")

    print(f"[load_checkpoint] Model ready on {device}. (epoch {loaded_epoch})")
    return model, loaded_epoch


# 10. TEST EVALUATION
def evaluate_test_set(model, test_dataset: TrOCRGraphemeDataset,
                      vocab: GraphemeVocab,
                      feature_extractor) -> pd.DataFrame:
    """
    Beam-search decoding (beam=4) on every test sample.
    Returns a DataFrame with columns:
        image_path | ground_truth | prediction | cer | correct
    """
    model.eval()
    model.to(device)

    all_preds, all_gts, all_paths = [], [], []

    print("\nRunning inference on test set    ")
    for idx in tqdm(range(len(test_dataset)), dynamic_ncols=True):
        path   = test_dataset.get_path(idx)
        gt_str = test_dataset.get_label(idx)

        img_bgr = cv2.imread(path)
        if img_bgr is None:
            continue
        img_rgb = Image.fromarray(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))

        pixel_values = feature_extractor(
            img_rgb, return_tensors='pt'
        ).pixel_values.to(device)

        with torch.no_grad():
            generated_ids = model.generate(
                pixel_values,
                max_new_tokens = MAX_LEN,
                num_beams      = 4,
                early_stopping = True,
            )

        pred = vocab.decode(generated_ids[0].cpu().tolist())
        all_preds.append(pred)
        all_gts.append(gt_str)
        all_paths.append(path)

    cer, wer = corpus_cer_wer(all_preds, all_gts)

    print(f"\n{'   '*55}")
    print(f"  EXPERIMENT  : TrOCR ViT  +  Grapheme Tokenizer")
    print(f"  Test samples: {len(all_gts):,}")
    print(f"  CER         : {cer  * 100:.2f} %")
    print(f"  WER         : {wer  * 100:.2f} %")
    print(f"  Word Acc    : {(1 - wer) * 100:.2f} %")
```

```
    print(f"{'   '*55}")

    results_df = pd.DataFrame({
        'image_path':   all_paths,
        'ground_truth': all_gts,
        'prediction':   all_preds,
        'cer':          [cer_single(p, g) for p, g in zip(all_preds, all_gts)],
        'correct':      [p == g           for p, g in zip(all_preds, all_gts)],
    })
    results_df.to_csv(RESULTS_CSV, index=False)
    print(f"Results saved     {RESULTS_CSV}\n")

    # print first 20 samples
    print(f"{'Ground Truth':<30} {'Prediction':<30} {'CER':>7}  OK")
    print("   " * 75)
    for _, row in results_df.head(20).iterrows():
        mark = "   " if row['correct'] else "   "
        print(f"{row['ground_truth']:<30} {row['prediction']:<30}"
              f" {row['cer']*100:>6.1f}%  {mark}")

    return results_df


# 11. ANALYSIS PLOTS
#     Identical plotting code across all 5 experiments so figures are
#     directly comparable.  Only titles/filenames differ.

def _read_trainer_history(trainer) -> dict:
    log = trainer.state.log_history
    history = {'train_loss': [], 'val_cer': [], 'val_wer': []}
    epoch_losses: dict = {}
    for entry in log:
        if 'loss' in entry and 'epoch' in entry:
            ep = int(entry['epoch'])
            epoch_losses.setdefault(ep, []).append(entry['loss'])
    eval_entries = [e for e in log if 'eval_cer' in e]
    for i, e in enumerate(eval_entries):
        ep = int(e.get('epoch', i + 1))
        avg_loss = float(np.mean(epoch_losses.get(ep, [float('nan')])))
        history['train_loss'].append(avg_loss)
        history['val_cer'].append(e['eval_cer'])
        history['val_wer'].append(e['eval_wer'])
    return history


def plot_training_curves(history: dict, save_dir=PLOT_DIR):
    if not history['val_cer']:
        print("No training history     skipping curve plot")
        return
    epochs = range(1, len(history['val_cer']) + 1)
    fig, axes = plt.subplots(1, 3, figsize=(15, 4), facecolor='#0f0f14')
    specs = [
        ('train_loss', 'Training Loss',    '#00d4ff'),
        ('val_cer',    'Validation CER',   '#ff6b6b'),
        ('val_wer',    'Validation WER',   '#ffd93d'),
    ]
    for ax, (key, title, color) in zip(axes, specs):
        vals = history[key]
        ax.set_facecolor('#1a1a24')
```

```
        ax.plot(epochs, vals, color=color, linewidth=2.5, zorder=3)
        ax.fill_between(epochs, vals, alpha=0.15, color=color, zorder=2)
        best_idx = int(np.nanargmin(vals))
        ax.scatter(best_idx + 1, vals[best_idx],
                   color=color, s=80, zorder=5,
                   edgecolors='white', linewidths=1.5)
        ax.annotate(f"best: {vals[best_idx]:.4f}",
                    xy=(best_idx + 1, vals[best_idx]),
                    xytext=(8, 8), textcoords='offset points',
                    color='white', fontsize=8)
        ax.set_title(title, color='white', fontsize=12, fontweight='bold')
        ax.set_xlabel('Epoch', color='#aaaaaa', fontsize=9)
        ax.tick_params(colors='#aaaaaa', labelsize=8)
        for spine in ax.spines.values():
            spine.set_edgecolor('#333344')
        ax.grid(color='#333344', linewidth=0.5, alpha=0.6)

    fig.suptitle('TrOCR ViT + Grapheme Tokenizer     Training Curves',
                 color='white', fontsize=13, fontweight='bold', y=1.02)
    plt.tight_layout(pad=2)
    path = f"{save_dir}/training_curves.png"
    plt.savefig(path, dpi=150, bbox_inches='tight', facecolor='#0f0f14')
    plt.close()
    print(f"Saved     {path}")


def plot_cer_distribution(results_df: pd.DataFrame, save_dir=PLOT_DIR):
    cer_vals = results_df['cer'].values * 100
    fig, axes = plt.subplots(1, 2, figsize=(14, 5), facecolor='#0f0f14')

    ax = axes[0]
    ax.set_facecolor('#1a1a24')
    bins = np.linspace(0, 100, 30)
    counts, edges, patches = ax.hist(cer_vals, bins=bins, edgecolor='none')
    for patch, left_edge in zip(patches, edges[:-1]):
        if left_edge == 0:   patch.set_facecolor('#2ecc71')
        elif left_edge < 20: patch.set_facecolor('#f39c12')
        else:                patch.set_facecolor('#e74c3c')
    ax.axvline(np.mean(cer_vals), color='white', linestyle='--',
               linewidth=1.5, label=f'Mean {np.mean(cer_vals):.1f}%')
    ax.axvline(np.median(cer_vals), color='#00d4ff', linestyle=':',
               linewidth=1.5, label=f'Median {np.median(cer_vals):.1f}%')
    ax.legend(facecolor='#1a1a24', labelcolor='white', fontsize=9)
    ax.set_xlabel('CER (%)', color='#aaaaaa')
    ax.set_ylabel('Count',   color='#aaaaaa')
    ax.set_title('CER Distribution', color='white', fontweight='bold')
    ax.tick_params(colors='#aaaaaa')
    for sp in ax.spines.values():
        sp.set_edgecolor('#333344')

    ax2 = axes[1]
    ax2.set_facecolor('#1a1a24')
    sorted_cer = np.sort(cer_vals)
    cumulative = np.arange(1, len(sorted_cer) + 1) / len(sorted_cer) * 100
    ax2.plot(sorted_cer, cumulative, color='#00d4ff', linewidth=2.5)
    ax2.fill_between(sorted_cer, cumulative, alpha=0.1, color='#00d4ff')
    for threshold, clr in [(0, '#2ecc71'), (10, '#f39c12'), (25, '#e74c3c')]:
        pct = np.interp(threshold, sorted_cer, cumulative)
        ax2.axvline(threshold, color=clr, linestyle='--', alpha=0.6, linewidth=1)
```

```
        ax2.text(threshold + 1, 5, f'{pct:.1f}%\ n  {threshold}%CER',
                 color=clr, fontsize=8)
    ax2.set_xlabel('CER (%)',                          color='#aaaaaa')
    ax2.set_ylabel('Cumulative % of samples',          color='#aaaaaa')
    ax2.set_title('Cumulative Accuracy vs CER',        color='white', fontweight='bold')
    ax2.tick_params(colors='#aaaaaa')
    for sp in ax2.spines.values():
        sp.set_edgecolor('#333344')
    ax2.grid(color='#333344', linewidth=0.5, alpha=0.6)

    fig.suptitle('TrOCR ViT + Grapheme Tokenizer     CER Distribution',
                 color='white', fontsize=13, fontweight='bold', y=1.02)
    plt.tight_layout(pad=2)
    path = f"{save_dir}/cer_distribution.png"
    plt.savefig(path, dpi=150, bbox_inches='tight', facecolor='#0f0f14')
    plt.close()
    print(f"Saved     {path}")


def plot_error_analysis(results_df: pd.DataFrame, save_dir=PLOT_DIR):
    df = results_df.copy()
    # word length in graphemes (linguistically meaningful for Bangla)
    df['gt_len'] = df['ground_truth'].apply(
        lambda w: len(segment_graphemes(str(w)))
    )
    buckets = [(1, 1), (2, 2), (3, 3), (4, 5), (6, 8), (9, 99)]
    labels_b, accs, counts = [], [], []
    for lo, hi in buckets:
        sub = df[(df['gt_len'] >= lo) & (df['gt_len'] <= hi)]
        if len(sub) == 0:
            continue
        lab = (f"{lo}" if lo == hi
               else (f"{lo}-{hi}" if hi < 99 else f"{lo}+"))
        labels_b.append(lab)
        accs.append(sub['correct'].mean() * 100)
        counts.append(len(sub))

    # confused grapheme pairs
    grapheme_pairs = Counter()
    for _, row in df[~df['correct']].iterrows():
        gt_gs   = segment_graphemes(str(row['ground_truth']))
        pred_gs = segment_graphemes(str(row['prediction']))
        for g, p in zip(gt_gs, pred_gs):
            if g != p:
                grapheme_pairs[(g, p)] += 1

    fig = plt.figure(figsize=(16, 6), facecolor='#0f0f14')
    gs  = gridspec.GridSpec(1, 2, figure=fig, wspace=0.35)

    ax1 = fig.add_subplot(gs[0])
    ax1.set_facecolor('#1a1a24')
    bar_colors = ['#2ecc71' if a >= 70 else '#f39c12' if a >= 40
                  else '#e74c3c' for a in accs]
    bars = ax1.bar(labels_b, accs, color=bar_colors, edgecolor='none', width=0.6)
    for bar, cnt in zip(bars, counts):
        ax1.text(bar.get_x() + bar.get_width() / 2,
                 bar.get_height() + 1,
                 f'n={cnt:,}', ha='center', va='bottom',
                 color='#aaaaaa', fontsize=8)
```

```
    ax1.set_xlabel('Word length (graphemes)', color='#aaaaaa')
    ax1.set_ylabel('Word Accuracy (%)',        color='#aaaaaa')
    ax1.set_title('Accuracy by Grapheme Count', color='white', fontweight='bold')
    ax1.set_ylim(0, 110)
    ax1.tick_params(colors='#aaaaaa')
    for sp in ax1.spines.values():
        sp.set_edgecolor('#333344')
    ax1.grid(axis='y', color='#333344', linewidth=0.5, alpha=0.6)

    ax2 = fig.add_subplot(gs[1])
    ax2.set_facecolor('#1a1a24')
    top_pairs = grapheme_pairs.most_common(15)
    if top_pairs:
        pair_labels = [f"{g}   {p}" for (g, p), _ in top_pairs]
        pair_counts = [c for _, c in top_pairs]
        y_pos = range(len(pair_labels))
        ax2.barh(list(y_pos), pair_counts, color='#ff6b6b',
                 edgecolor='none', height=0.6)
        ax2.set_yticks(list(y_pos))
        ax2.set_yticklabels(pair_labels, color='white', fontsize=9)
        ax2.invert_yaxis()
        ax2.set_xlabel('Frequency', color='#aaaaaa')
        ax2.set_title('Top 15 Confused Grapheme Pairs',
                      color='white', fontweight='bold')
        ax2.tick_params(colors='#aaaaaa')
        for sp in ax2.spines.values():
            sp.set_edgecolor('#333344')
        ax2.grid(axis='x', color='#333344', linewidth=0.5, alpha=0.6)

    fig.suptitle('TrOCR ViT + Grapheme     Error Analysis',
                 color='white', fontsize=14, fontweight='bold', y=1.02)
    path = f"{save_dir}/error_analysis.png"
    plt.savefig(path, dpi=150, bbox_inches='tight', facecolor='#0f0f14')
    plt.close()
    print(f"Saved     {path}")


def plot_tsne(model, test_dataset: TrOCRGraphemeDataset,
              feature_extractor, n_samples=600, save_dir=PLOT_DIR):
    """
    Extract ViT CLS-token features and project with t-SNE,
    coloured by word-length bucket (in graphemes).
    """
    model.eval()
    model.to(device)

    idxs = random.sample(range(len(test_dataset)),
                         min(n_samples, len(test_dataset)))
    feats, lengths = [], []

    with torch.no_grad():
        for idx in tqdm(idxs, desc="Extracting ViT features for t-SNE",
                        dynamic_ncols=True):
            path = test_dataset.get_path(idx)
            gt   = test_dataset.get_label(idx)
            img_bgr = cv2.imread(path)
            if img_bgr is None:
                continue
            img_rgb = Image.fromarray(
```

```
                cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
            )
            pv = feature_extractor(
                img_rgb, return_tensors='pt'
            ).pixel_values.to(device)

            # ViT encoder: last_hidden_state shape (1, 577, 768) for base
            enc_out  = model.encoder(pixel_values=pv)
            cls_feat = enc_out.last_hidden_state[:, 0, :].squeeze(0)
            feats.append(cls_feat.cpu().float().numpy())
            lengths.append(len(segment_graphemes(gt)))

    feats   = np.array(feats)
    lengths = np.array(lengths)

    print("Running t-SNE    ")
    tsne  = TSNE(n_components=2, perplexity=30, n_iter=1000,
                 random_state=Config.SEED)
    emb2d = tsne.fit_transform(feats)

    def bucket(l):
        if l == 1: return '1'
        if l == 2: return '2'
        if l == 3: return '3'
        if l <= 5: return '4-5'
        if l <= 8: return '6-8'
        return '9+'

    buckets = [bucket(l) for l in lengths]
    palette = {'1': '#2ecc71', '2': '#3498db', '3': '#9b59b6',
               '4-5': '#f39c12', '6-8': '#e74c3c', '9+': '#95a5a6'}

    fig, ax = plt.subplots(figsize=(10, 8), facecolor='#0f0f14')
    ax.set_facecolor('#1a1a24')
    for label, color in palette.items():
        mask = np.array(buckets) == label
        if mask.sum() == 0:
            continue
        ax.scatter(emb2d[mask, 0], emb2d[mask, 1],
                   c=color, label=f'{label} grapheme(s)',
                   alpha=0.65, s=20, edgecolors='none')
    ax.legend(facecolor='#1a1a24', labelcolor='white',
              title='Word length (graphemes)', title_fontsize=9,
              fontsize=9, loc='upper right')
    ax.set_title(
        'TrOCR ViT + Grapheme     t-SNE of ViT CLS features',
        color='white', fontsize=13, fontweight='bold'
    )
    ax.tick_params(colors='#aaaaaa')
    for sp in ax.spines.values():
        sp.set_edgecolor('#333344')

    path = f"{save_dir}/tsne_features.png"
    plt.savefig(path, dpi=150, bbox_inches='tight', facecolor='#0f0f14')
    plt.close()
    print(f"Saved     {path}")


def plot_error_grid(results_df: pd.DataFrame, max_errors=40,
```

```
                  save_dir=PLOT_DIR):
    """Render erroneous predictions as an image grid (requires Bengali font)."""
    import subprocess
    subprocess.run(["apt-get", "install", "-y", "fonts-beng", "-qq"],
                   capture_output=True)

    def find_bangla_font():
        for pat in ["/usr/share/fonts/**/*Lohit*Bengali*",
                    "/usr/share/fonts/**/*Lohit*Beng*",
                    "/usr/share/fonts/**/*Bengali*"]:
            hits = glob.glob(pat, recursive=True)
            if hits:
                return hits[0]
        return None

    font_path = find_bangla_font()
    if font_path is None:
        print("Bengali font not found     skipping error grid")
        return

    COLS   = 4
    CELL_W = 320
    IMG_H  = 80
    TEXT_H = 70
    CELL_H = IMG_H + TEXT_H
    font   = ImageFont.truetype(font_path, 18)
    sfont  = ImageFont.truetype(font_path, 13)

    errors = (results_df[~results_df['correct']]
              .head(max_errors).reset_index(drop=True))
    print(f"\nError grid: {len(errors)} samples")

    def make_cell(i, row):
        cell = Image.new("RGB", (CELL_W, CELL_H), (245, 245, 245))
        draw = ImageDraw.Draw(cell)
        raw  = cv2.imread(row['image_path'], cv2.IMREAD_GRAYSCALE)
        if raw is not None:
            scale  = min(IMG_H / raw.shape[0], (CELL_W - 10) / raw.shape[1])
            rh, rw = int(raw.shape[0] * scale), int(raw.shape[1] * scale)
            raw    = cv2.resize(raw, (rw, rh))
            pil    = Image.fromarray(raw).convert("RGB")
            cell.paste(pil, ((CELL_W - rw) // 2, (IMG_H - rh) // 2))
        draw.rectangle([0, 0, 26, 20], fill=(50, 50, 50))
        draw.text((4, 2), f"#{i+1}", font=sfont, fill=(255, 255, 255))
        border = ((220, 50, 50) if row['cer'] > 0.5
                  else (255, 165, 0) if row['cer'] > 0.2
                  else (50, 180, 50))
        draw.rectangle([0, 0, CELL_W - 1, CELL_H - 1],
                       outline=border, width=3)
        draw.line([(0, IMG_H), (CELL_W, IMG_H)],
                  fill=(180, 180, 180), width=1)
        draw.text((6,  IMG_H + 4),  "GT:", font=sfont, fill=(30,  120, 30))
        draw.text((36, IMG_H + 2),  row['ground_truth'],
                  font=font, fill=(20, 100, 20))
        draw.text((6,  IMG_H + 30), "PR:", font=sfont, fill=(160, 40,  40))
        draw.text((36, IMG_H + 28), row['prediction'],
                  font=font, fill=(140, 20, 20))
        draw.text((CELL_W - 44, IMG_H + 4),
                  f"{row['cer']*100:.0f}%", font=sfont, fill=(80, 80, 80))
```

```
        return cell

    cells = [make_cell(i, row) for i, row in errors.iterrows()]
    ROWS  = math.ceil(len(cells) / COLS)
    grid  = Image.new("RGB", (CELL_W * COLS, CELL_H * ROWS), (210, 210, 210))
    for idx, cell in enumerate(cells):
        r, c = divmod(idx, COLS)
        grid.paste(cell, (c * CELL_W, r * CELL_H))

    path = f"{save_dir}/error_grid.png"
    grid.save(path, dpi=(150, 150))

    fig, ax = plt.subplots(figsize=(COLS * 3.8, ROWS * 2.4))
    ax.imshow(grid)
    ax.axis('off')
    ax.set_title("Erroneous Predictions  "
                 "(green <20% CER | orange 20-50% | red >50%)",
                 fontsize=12, pad=10)
    plt.tight_layout()
    plt.savefig(f"{save_dir}/error_grid_plot.png",
                dpi=150, bbox_inches='tight')
    plt.close()
    print(f"Saved     {path}")


def print_summary(results_df: pd.DataFrame, history: dict):
    cer_vals = results_df['cer'].values
    wer      = 1 - results_df['correct'].mean()

    print(f"\n{'   '*60}")
    print(f"  EXPERIMENT  : TrOCR ViT  +  Grapheme Tokenizer")
    print(f"{'   '*60}")
    print(f"  Test samples          : {len(results_df):,}")
    print(f"  Word Accuracy         : {results_df['correct'].mean()*100:.2f} %")
    print(f"  Corpus CER            : {cer_vals.mean()*100:.2f} %")
    print(f"  Corpus WER            : {wer*100:.2f} %")
    print(f"  Median CER            : {np.median(cer_vals)*100:.2f} %")
    print(f"  CER = 0  (perfect)    : {(cer_vals==0).sum()/len(cer_vals)*100:.1f} %")
    print(f"  CER     0.1 (near-perf) : {(cer_vals<=0.1).sum()/len(cer_vals)*100:.1f} %")
    print(f"  CER > 0.5 (bad)       : {(cer_vals>0.5).sum()/len(cer_vals)*100:.1f} %")
    if history and history['val_cer']:
        print(f"  Best Val CER (train)  : {min(history['val_cer'])*100:.2f} %")
    print(f"{'   '*60}\n")



if __name__ == '__main__':

    df = load_dataset(ROOT)

    train_val_df, test_df = train_test_split(
        df,
        test_size    = Config.TEST_SPLIT,
        random_state = Config.SEED,
    )
    val_frac = Config.VAL_SPLIT / (1.0 - Config.TEST_SPLIT)
    train_df, val_df = train_test_split(
        train_val_df,
        test_size    = val_frac,
```

```
        random_state = Config.SEED,
    )
    train_df = train_df.reset_index(drop=True)
    val_df   = val_df.reset_index(drop=True)
    test_df  = test_df.reset_index(drop=True)
    print(f"Train: {len(train_df):,} | Val: {len(val_df):,} | "
          f"Test: {len(test_df):,}")

    vocab = GraphemeVocab().build(train_df['label'].dropna())
    vocab.save(f'{CKPT_DIR}/grapheme_vocab.json')

    feature_extractor = AutoImageProcessor.from_pretrained(Config.TROCR_MODEL)


    def _has_epoch_folders(d):
        return any(os.path.isdir(f)
                   for f in glob.glob(os.path.join(d, "epoch_*")))

    if _has_epoch_folders(CKPT_DIR) or _has_epoch_folders(PREV_CKPT_DIR):
        # Load vocab from the read-only input (it was built on the training set
        # and must stay identical across runs).
        vocab = GraphemeVocab.load(
            os.path.join(PREV_CKPT_DIR, "grapheme_vocab.json")
        )
        model, start_epoch = load_checkpoint(vocab=vocab)
        print(f"Resuming: loaded epoch {start_epoch}, "
              f"next checkpoint     epoch_{start_epoch+1:02d}")
    else:
        start_epoch = 0
        model = build_model(vocab)
        print("No prior checkpoints found     starting fresh from epoch 0")

    train_dataset = TrOCRGraphemeDataset(train_df, vocab, feature_extractor)
    val_dataset   = TrOCRGraphemeDataset(val_df,   vocab, feature_extractor)
    test_dataset  = TrOCRGraphemeDataset(test_df,  vocab, feature_extractor)

    trainer = train_model(model, train_dataset, val_dataset, vocab,
                          start_epoch=start_epoch)
    best_ckpt = trainer.state.best_model_checkpoint
    if best_ckpt:
        print(f"\nLoading best checkpoint from {best_ckpt}    ")
        model = VisionEncoderDecoderModel.from_pretrained(best_ckpt).to(device)
        # re-apply generation config after loading from disk
        model.generation_config.decoder_start_token_id = BOS_ID
        model.generation_config.pad_token_id           = PAD_ID
        model.generation_config.eos_token_id           = EOS_ID
        model.generation_config.max_new_tokens          = MAX_LEN
    else:
        print("No best checkpoint found     using final weights")

    results_df = evaluate_test_set(model, test_dataset, vocab, feature_extractor)

    history = _read_trainer_history(trainer)
    print_summary(results_df, history)

    print(f"\nGenerating analysis plots     {PLOT_DIR}/\n")
    plot_training_curves(history)
    plot_cer_distribution(results_df)
    plot_error_analysis(results_df)
```

```
    plot_tsne(model, test_dataset, feature_extractor, n_samples=600)
    plot_error_grid(results_df, max_errors=40)

    print("\nAll done.")
```

A.2 Synthetic Scene text Generation

```
"""
Bengali Scene Text     Calibrated Synthetic Generator
=====================================================
All augmentation parameters are calibrated from real dataset analysis:

Real dataset stats used:
  BG colors       : 8 K-means clusters from real images
  Text colors     : 6 K-means clusters from real images
  Text/BG style   : 43.2% dark-on-light, 56.8% light-on-dark
  Contrast ratio  : median=1.12 (very low     faded/weathered look)
  Font size proxy : p5=12, p95=170  (wide range)
  Aspect ratio    : 59.5% wide (1.5  3  ), 21.2% squarish, 12.7% very wide
  Padding         : median=0.079 (very tight crops)
  Skew            : 60% near-flat, 27.6% mild (3   10   ), 12.3% heavy (>10  )
  Sharpness       : 18.8% blurry, 38.2% medium, 43% sharp
  Blur type       : 86.7% no blur, 13% gaussian/defocus, 0.3% motion
  Noise level     : p5=2.9, p95=34.8
  JPEG blockiness : p25=7.7, p75=23.3      quality floor lowered
  Grayscale       : 23.1% of real images
  Degradation     : 23.6% clean, 62.6% mild, 21.5% heavy
"""

import os
import cv2
import csv
import json
import math
import random
import numpy as np
from PIL import Image, ImageDraw, ImageFont
from pathlib import Path
from typing import Tuple, List
import warnings
warnings.filterwarnings('ignore')

# ============================================================================
# CONFIGURATION
# ============================================================================

WORDS_FILE = '/kaggle/input/datasets/kaidul007/syn-scene-text-words/syn_scene_text (1).txt'
FONT_ROOT  = Path('/kaggle/input/datasets/kayed007/my-fonts/all fonts')
OUTPUT_DIR = Path('/kaggle/working/synthetic_scene_text')
NUM_IMAGES = 20000

# Layout probabilities (unchanged)
LAYOUT_STRAIGHT = 0.90
LAYOUT_CIRCULAR = 0.05
LAYOUT_SINUSOID = 0.05

# ============================================================================
# CALIBRATED COLOR PALETTE
```

```
# (derived from K-means clusters on real dataset)
# ============================================================================

# Background clusters with their real-world frequency weights
# Format: (R, G, B, weight)
BG_CLUSTERS = [
    # cluster 2     light gray-blue       19.4%
    ((179, 179, 184), 0.194),
    # cluster 5     very light warm white 18.4%
    ((224, 224, 221), 0.184),
    # cluster 0     mid warm brown        15.9%
    ((156, 136, 125), 0.159),
    # cluster 7     mid dark gray         11.1%
    (( 88,  85,  83), 0.111),
    # cluster 3     mid dark blue          9.7%
    (( 44,  97, 158), 0.097),
    # cluster 1     dark near-black        9.2%
    (( 32,  31,  39), 0.092),
    # cluster 4     light golden yellow    9.3%
    ((219, 178,  77), 0.093),
    # cluster 6     mid dark red           7.0%
    ((163,  52,  47), 0.070),
]

BG_COLORS_LIST   = [c[0] for c in BG_CLUSTERS]
BG_WEIGHTS       = [c[1] for c in BG_CLUSTERS]

# Text color clusters from real dataset
TEXT_CLUSTERS = [
    ((229, 229, 227), 0.288),   # near-white      28.8%
    ((164, 163, 167), 0.244),   # mid gray        24.4%
    (( 37,  32,  37), 0.140),   # near-black      14.0%
    ((142,  73,  64), 0.127),   # warm red-brown  12.7%
    (( 42,  92, 143), 0.107),   # mid blue        10.7%
    ((218, 175,  77), 0.094),   # warm gold        9.4%
]

TEXT_COLORS_LIST = [c[0] for c in TEXT_CLUSTERS]
TEXT_WEIGHTS     = [c[1] for c in TEXT_CLUSTERS]

def _weighted_choice(items, weights):
    r = random.random()
    acc = 0.0
    for item, w in zip(items, weights):
        acc += w
        if r < acc:
            return item
    return items[-1]

# ============================================================================
# CALIBRATED COLOR PICKER
# Real dataset: 43.2% dark-on-light, 56.8% light-on-dark
# Contrast median = 1.12     we simulate low-contrast faded text,
# but always enforce a minimum luminance gap so text stays visible.
# ============================================================================

def luminance_rgb(r, g, b):
    return 0.299 * r + 0.587 * g + 0.114 * b

```

```python
# Minimum luminance difference between text and background.
# Below this the text is effectively invisible.
# 30 out of 255 (~12%)     readable but can look faded.
MIN_LUM_GAP = 45

def _enforce_min_gap(text, bg, dark_on_light, min_gap=MIN_LUM_GAP):
    """
    If text and bg are too close in luminance, push text further away
    until the gap is at least min_gap. Clamps to [0, 255].
    """
    t_lum = luminance_rgb(*text)
    b_lum = luminance_rgb(*bg)
    gap   = b_lum - t_lum if dark_on_light else t_lum - b_lum

    if gap >= min_gap:
        return text

    deficit = min_gap - gap
    if dark_on_light:
        # text needs to be darker     scale channels down
        target_lum = max(0, t_lum - deficit)
        scale      = target_lum / max(t_lum, 1)
        text       = tuple(int(np.clip(c * scale, 0, 255)) for c in text)
    else:
        # text needs to be brighter     push channels toward 255
        boost = deficit / max(255 - t_lum, 1)
        text  = tuple(int(np.clip(c + (255 - c) * boost, 0, 255)) for c in text)

    return text

def pick_bg_and_text_color() -> Tuple[Tuple, Tuple]:
    """
    1. Sample bg from real cluster distribution (with small jitter).
    2. Decide dark-on-light (43%) vs light-on-dark (56%) to match real ratio.
    3. Sample text color from real cluster, enforce correct direction.
    4. Optionally apply low-contrast blend (faded/weathered look),
       but always clamp so luminance gap stays >= MIN_LUM_GAP.

    Contrast tiers (matching real degradation distribution):
      ~35% normal contrast      no blend applied
      ~45% mild fade            blend 15  35  % toward bg
      ~20% heavy fade           blend 36  55  % toward bg
    Old ceiling was 0.65 blend which caused invisible text; now capped at
    0.55 and _enforce_min_gap provides a hard floor regardless.
    """
    # --- background ---
    bg_base = _weighted_choice(BG_COLORS_LIST, BG_WEIGHTS)
    jitter  = lambda: random.randint(-18, 18)
    bg = tuple(int(np.clip(c + jitter(), 0, 255)) for c in bg_base)

    bg_lum  = luminance_rgb(*bg)

    # --- decide style ---
    dark_on_light = random.random() < 0.432   # 43.2% from real data

    # --- sample text color from real clusters ---
    text_base = _weighted_choice(TEXT_COLORS_LIST, TEXT_WEIGHTS)
    text_lum  = luminance_rgb(*text_base)

```

```
    # Enforce direction: retry up to 6 times to get a valid candidate
    for _ in range(6):
        if dark_on_light and text_lum < bg_lum:
            break
        if not dark_on_light and text_lum > bg_lum:
            break
        text_base = _weighted_choice(TEXT_COLORS_LIST, TEXT_WEIGHTS)
        text_lum  = luminance_rgb(*text_base)
    else:
        # Fallback: guaranteed correct polarity
        if dark_on_light:
            text_base = (30, 28, 30)
        else:
            text_base = (225, 225, 223)

    text_jitter = lambda: random.randint(-12, 12)
    text = tuple(int(np.clip(c + text_jitter(), 0, 255)) for c in text_base)

    # --- contrast tier ---
    r = random.random()
    if r < 0.35:
        pass                                          # normal     no blend
    elif r < 0.80:
        blend = random.uniform(0.15, 0.35)            # mild fade
        text  = tuple(int(t * (1 - blend) + b * blend) for t, b in zip(text, bg))
    else:
        blend = random.uniform(0.36, 0.55)            # heavy fade (was up to 0.65)
        text  = tuple(int(t * (1 - blend) + b * blend) for t, b in zip(text, bg))

    # --- hard floor: always guarantee minimum visible gap ---
    text = _enforce_min_gap(text, bg, dark_on_light)

    return bg, text


# ============================================================================
# FONT DISCOVERY & VALIDATION
# ============================================================================

def discover_fonts(font_root: Path) -> List[str]:
    fonts = sorted(font_root.glob('*/Unicode/*.ttf'))
    print(f"    Found {len(fonts)} fonts under {font_root}")
    return [str(f) for f in fonts]

def font_renders_bengali(font_path: str, size: int = 40) -> bool:
    try:
        font = ImageFont.truetype(font_path, size)
    except Exception:
        return False
    img  = Image.new('L', (80, 80), 255)
    draw = ImageDraw.Draw(img)
    draw.text((10, 10), '    ', font=font, fill=0)
    arr  = np.array(img)
    return np.sum(arr < 200) >= 30

print("Discovering fonts...")
ALL_FONTS   = discover_fonts(FONT_ROOT)
VALID_FONTS = [f for f in ALL_FONTS if font_renders_bengali(f)]
print(f"    {len(VALID_FONTS)} / {len(ALL_FONTS)} fonts valid")
```

```
if not VALID_FONTS:
    raise RuntimeError("No valid Bengali fonts found!")

# ============================================================================
# WORD LIST
# ============================================================================

def load_words(filepath: str) -> List[str]:
    with open(filepath, 'r', encoding='utf-8') as f:
        words = [l.strip() for l in f if l.strip() and len(l.strip()) >= 2]
    print(f"    {len(words)} words loaded from {filepath}")
    return words

WORDS = load_words(WORDS_FILE)

# ============================================================================
# CALIBRATED FONT SIZE PICKER
# Real: font_size_proxy p5=12, p95=170
# Strategy: sample from a mixture     most images are smaller,
# with a long tail for large text.
# ============================================================================

def pick_font_size() -> int:
    """
    Mixture distribution matching real font_size_proxy:
      60% small  (16  55   px)      most real crops are small words
      30% medium (56  100   px)     mid-range
      10% large  (101  170   px)    large signage
    """
    r = random.random()
    if r < 0.60:
        return random.randint(16, 55)
    elif r < 0.90:
        return random.randint(56, 100)
    else:
        return random.randint(101, 170)

# ============================================================================
# CALIBRATED PADDING PICKER
# Real: padding_ratio median=0.079     very tight crops
# ============================================================================

def pick_padding():
    """
    Very tight crops matching real dataset (median padding ~8%).
    Returns (pad_x, pad_y) in pixels for a given font size.
    """
    pad_x = random.randint(2, 12)    # was 8  25
    pad_y = random.randint(2,  8)    # was 8  20
    return pad_x, pad_y

# ============================================================================
# CORE RENDERER     straight layout (calibrated)
# ============================================================================

def render_word_straight(word, font_path, bg_color, text_color):
    font_size = pick_font_size()
    try:
        font = ImageFont.truetype(font_path, font_size)
```

```
    except Exception:
        font = ImageFont.load_default()

    dummy = Image.new('RGB', (1, 1))
    draw  = ImageDraw.Draw(dummy)
    bbox  = draw.textbbox((0, 0), word, font=font)
    tw    = bbox[2] - bbox[0]
    th    = bbox[3] - bbox[1]

    pad_x, pad_y = pick_padding()
    W = tw + 2 * pad_x
    H = th + 2 * pad_y

    img  = Image.new('RGB', (W, H), bg_color)
    draw = ImageDraw.Draw(img)
    draw.text((pad_x - bbox[0], pad_y - bbox[1]), word,
              font=font, fill=text_color)
    return img

# ============================================================================
# PATH-WARP HELPERS (unchanged)
# ============================================================================

def crop_to_content(img: Image.Image,
                    bg_color: Tuple,
                    margin: int = 4) -> Image.Image:    # margin reduced: 8  4
    arr = np.array(img)
    bg  = np.array(bg_color, dtype=np.float32)
    diff = np.abs(arr.astype(np.float32) - bg).max(axis=2)
    mask = diff > 15
    rows = np.any(mask, axis=1)
    cols = np.any(mask, axis=0)
    if not rows.any():
        return img
    r0, r1 = np.where(rows)[0][[0, -1]]
    c0, c1 = np.where(cols)[0][[0, -1]]
    H, W   = arr.shape[:2]
    r0 = max(0, r0 - margin)
    r1 = min(H - 1, r1 + margin)
    c0 = max(0, c0 - margin)
    c1 = min(W - 1, c1 + margin)
    return img.crop((c0, r0, c1 + 1, r1 + 1))

# ============================================================================
# CIRCULAR LAYOUT (unchanged logic, calibrated font size)
# ============================================================================

def render_word_circular(word, font_path, bg_color, text_color):
    font_size = pick_font_size()
    try:
        font = ImageFont.truetype(font_path, font_size)
    except Exception:
        font = ImageFont.load_default()

    dummy = Image.new('RGB', (1, 1))
    draw  = ImageDraw.Draw(dummy)
    bbox  = draw.textbbox((0, 0), word, font=font)
    tw    = bbox[2] - bbox[0]
    th    = bbox[3] - bbox[1]
```

```
347
348     pad_x, pad_y = pick_padding()
349     SW = tw + 2 * pad_x
350     SH = th + 2 * pad_y
351
352     src_img = Image.new('RGB', (SW, SH), bg_color)
353     draw    = ImageDraw.Draw(src_img)
354     draw.text((pad_x - bbox[0], pad_y - bbox[1]), word,
355               font=font, fill=text_color)
356     src = np.array(src_img)
357
358     arc_fraction = random.uniform(0.25, 0.50)
359     arc_angle    = arc_fraction * 2 * math.pi
360     radius       = SW / arc_angle
361
362     canvas_r = int(radius + SH * 2)
363     OW = OH  = 2 * canvas_r
364     cx = cy  = canvas_r
365     top_arc  = random.choice([True, False])
366
367     oy_coords, ox_coords = np.mgrid[0:OH, 0:OW]
368     dx = ox_coords - cx
369     dy = oy_coords - cy
370     r_out = np.sqrt(dx**2 + dy**2)
371     theta = np.arctan2(dy, dx)
372
373     base_angle  = -math.pi / 2 if top_arc else math.pi / 2
374     theta_start = base_angle - arc_angle / 2
375     theta_end   = base_angle + arc_angle / 2
376     theta_norm  = (theta - theta_start) / arc_angle
377
378     src_x = theta_norm * SW if top_arc else (1.0 - theta_norm) * SW
379     src_y = ((radius + SH) - r_out) if top_arc else (r_out - radius)
380
381     in_angle  = (theta >= theta_start) & (theta <= theta_end)
382     in_radial = (r_out >= radius) & (r_out <= radius + SH)
383     valid     = in_angle & in_radial
384
385     map_x = np.where(valid, src_x, -1).astype(np.float32)
386     map_y = np.where(valid, src_y, -1).astype(np.float32)
387
388     warped = cv2.remap(src.astype(np.float32), map_x, map_y,
389                        interpolation=cv2.INTER_LINEAR,
390                        borderMode=cv2.BORDER_CONSTANT,
391                        borderValue=list(bg_color))
392     return crop_to_content(Image.fromarray(warped.astype(np.uint8)), bg_color)
393
394
395 # ================================================================================
396 # SINUSOIDAL LAYOUT (unchanged logic, calibrated font size)
397 # ================================================================================
398
399 def render_word_sinusoidal(word, font_path, bg_color, text_color):
400     font_size = pick_font_size()
401     try:
402         font = ImageFont.truetype(font_path, font_size)
403     except Exception:
404         font = ImageFont.load_default()
405
```

```
    dummy = Image.new('RGB', (1, 1))
    draw  = ImageDraw.Draw(dummy)
    bbox  = draw.textbbox((0, 0), word, font=font)
    tw    = bbox[2] - bbox[0]
    th    = bbox[3] - bbox[1]

    pad_x, pad_y = pick_padding()
    SW = tw + 2 * pad_x
    SH = th + 2 * pad_y

    src_img = Image.new('RGB', (SW, SH), bg_color)
    draw    = ImageDraw.Draw(src_img)
    draw.text((pad_x - bbox[0], pad_y - bbox[1]), word,
              font=font, fill=text_color)
    src = np.array(src_img)

    amplitude  = random.uniform(0.04, 0.12) * SH
    n_cycles   = random.uniform(0.5, 1.2)
    wavelength = SW / n_cycles
    phase      = random.uniform(0, 2 * math.pi)

    A  = int(math.ceil(amplitude))
    OW = SW
    OH = SH + 2 * A

    oy_coords, ox_coords = np.mgrid[0:OH, 0:OW]
    offset = amplitude * np.sin(2 * math.pi * ox_coords / wavelength + phase)
    map_x  = ox_coords.astype(np.float32)
    map_y  = (oy_coords - A - offset).astype(np.float32)

    warped = cv2.remap(src.astype(np.float32), map_x, map_y,
                       interpolation=cv2.INTER_LINEAR,
                       borderMode=cv2.BORDER_CONSTANT,
                       borderValue=list(bg_color))
    return crop_to_content(Image.fromarray(warped.astype(np.uint8)), bg_color)


# ============================================================================
# DISPATCH
# ============================================================================

def render_word(word, font_path, bg_color, text_color):
    r = random.random()
    if r < LAYOUT_CIRCULAR:
        return render_word_circular(word, font_path, bg_color, text_color), 'circular'
    elif r < LAYOUT_CIRCULAR + LAYOUT_SINUSOID:
        return render_word_sinusoidal(word, font_path, bg_color, text_color), 'sinusoidal'
    else:
        return render_word_straight(word, font_path, bg_color, text_color), 'straight'


# ============================================================================
# NEIGHBORING TEXT BLEED
# Bleed text is always positioned so it starts OUTSIDE the canvas edge,
# meaning only a partial slice is visible at the border     never overlapping
# the main word in the centre.
#
# Rule per side:
#   LEFT  bleed: text drawn at x = -(tw - overlap), where overlap     [1, 8px]
```

```
#                so the right edge of the bleed word just kisses the left border
#   RIGHT bleed: text drawn at x = W - overlap
#                so only the first few pixels of the bleed word are visible
#   TOP   bleed: y chosen so baseline is above y=0 (text entirely above canvas)
#                but the descenders/bottom of glyphs may clip in by 1  4px
#   BOTTOM bleed: y = H + random small offset (text starts below canvas)
# ================================================================================

def add_text_bleed(img, font_path, bg_color):
    draw      = ImageDraw.Draw(img)
    W, H      = img.size
    font_size = pick_font_size()
    try:
        font = ImageFont.truetype(font_path, font_size)
    except Exception:
        return img

    brightness = 0.299*bg_color[0] + 0.587*bg_color[1] + 0.114*bg_color[2]
    if brightness < 128:
        bleed_color = tuple(int(np.clip(c + random.randint(30, 80), 0, 255))
                            for c in bg_color)
    else:
        bleed_color = tuple(int(np.clip(c - random.randint(30, 80), 0, 255))
                            for c in bg_color)

    # LEFT bleed     only the right edge of the neighbour peeks in
    if random.random() < 0.5:
        neighbor = random.choice(WORDS)
        bbox     = draw.textbbox((0, 0), neighbor, font=font)
        tw       = bbox[2] - bbox[0]
        th       = bbox[3] - bbox[1]
        overlap  = random.randint(1, 8)          # px visible inside canvas
        x        = -(tw - overlap) - bbox[0]     # anchor so right edge = overlap
        y        = random.randint(0, max(0, H - th))
        draw.text((x, y), neighbor, font=font, fill=bleed_color)

    # RIGHT bleed     only the left edge of the neighbour peeks in
    if random.random() < 0.5:
        neighbor = random.choice(WORDS)
        bbox     = draw.textbbox((0, 0), neighbor, font=font)
        th       = bbox[3] - bbox[1]
        overlap  = random.randint(1, 8)          # px visible inside canvas
        x        = W - overlap - bbox[0]         # anchor so left edge = W-overlap
        y        = random.randint(0, max(0, H - th))
        draw.text((x, y), neighbor, font=font, fill=bleed_color)

    # TOP or BOTTOM bleed     only a sliver of the neighbour's edge is visible
    if random.random() < 0.25:
        neighbor = random.choice(WORDS)
        bbox     = draw.textbbox((0, 0), neighbor, font=font)
        th       = bbox[3] - bbox[1]
        x        = random.randint(-20, max(0, W // 3))

        if random.random() < 0.5:
            # TOP: position so only bottom 1  4px   of glyph bleeds into canvas
            overlap = random.randint(1, 4)
            y       = -(th - overlap) - bbox[1]
        else:
            # BOTTOM: position so only top 1  4px   of glyph bleeds into canvas
```

```
            overlap = random.randint(1, 4)
            y       = H - overlap - bbox[1]

        draw.text((x, y), neighbor, font=font, fill=bleed_color)

    return img


# ============================================================================
# BARS / LINE ARTIFACTS (unchanged)
# ============================================================================

def add_bars(img, bg_color):
    draw = ImageDraw.Draw(img)
    W, H = img.size
    base_val = random.randint(80, 160)
    color    = (base_val,) * 3
    for _ in range(random.randint(1, 4)):
        x = random.randint(0, W - 1)
        draw.line([(x, 0), (x, H)], fill=color, width=random.randint(1, 2))
    for _ in range(random.randint(0, 2)):
        y = random.randint(0, H - 1)
        draw.line([(0, y), (W, y)], fill=color, width=random.randint(1, 2))
    return img


# ============================================================================
# CALIBRATED ROTATION
# Real: 60% near-flat (<3  ), 27.6% mild (3   10   ), 12.3% heavy (>10  )
# Old code: 10% chance of   25  , else   5       too much rotation
# ============================================================================

def apply_rotation(img):
    r = random.random()
    if r < 0.601:                           # 60% near-flat
        angle = random.uniform(-2.5, 2.5)
    elif r < 0.877:                         # 27.6% mild
        angle = random.uniform(-10, 10)
        if abs(angle) < 3:
            angle = math.copysign(3 + random.uniform(0, 7), angle)
    else:                                   # 12.3% heavy
        angle = random.uniform(-25, 25)
        if abs(angle) < 10:
            angle = math.copysign(10 + random.uniform(0, 15), angle)

    bg = img.getpixel((0, 0))
    return img.rotate(angle, expand=True, resample=Image.BICUBIC, fillcolor=bg)


# ============================================================================
# PERSPECTIVE (unchanged)
# ============================================================================

def apply_perspective(img):
    arr      = np.array(img)
    H, W     = arr.shape[:2]
    strength = random.uniform(0.03, 0.10)
    src = np.float32([[0,0],[W,0],[W,H],[0,H]])

```

```
    def jitter():
        return random.uniform(-strength, strength)

    dst = np.float32([
        [W * jitter(),       H * jitter()],
        [W * (1+jitter()),   H * jitter()],
        [W * (1+jitter()),   H * (1+jitter())],
        [W * jitter(),       H * (1+jitter())],
    ])
    M   = cv2.getPerspectiveTransform(src, dst)
    bg  = [int(img.getpixel((0,0))[c]) for c in range(3)]
    out = cv2.warpPerspective(arr, M, (W, H),
                              borderMode=cv2.BORDER_CONSTANT,
                              borderValue=bg)
    return Image.fromarray(out)


# ============================================================================
# CALIBRATED BLUR
# Real: 86.7% no blur, 13% gaussian/defocus, 0.3% motion
# Old code: 80% chance of blur     way too high
# ============================================================================

def apply_blur(img):
    r = random.random()

    if r < 0.867:                          # 86.7% sharp     no blur
        return img

    elif r < 0.997:                        # 13% gaussian / defocus
        arr   = np.array(img)
        sigma = random.uniform(0.4, 2.5)   # mild blur (real images not heavily blurred)
        ksize = int(sigma * 3) * 2 + 1
        arr   = cv2.GaussianBlur(arr, (ksize, ksize), sigma)
        return Image.fromarray(arr.astype(np.uint8))

    else:                                  # 0.3% motion
        arr   = np.array(img)
        size  = random.choice([3, 5, 7])
        angle = random.uniform(0, 180)
        k     = np.zeros((size, size))
        k[size//2, :] = 1
        k     = k / size
        M     = cv2.getRotationMatrix2D((size//2, size//2), angle, 1)
        k     = cv2.warpAffine(k, M, (size, size))
        arr   = cv2.filter2D(arr, -1, k)
        return Image.fromarray(arr.astype(np.uint8))


# ============================================================================
# CALIBRATED NOISE
# Real: noise_level p5=2.9, p95=34.8, median=15.6
# Old code: uniform(2, 12)     std too low
# ============================================================================

def apply_noise(img):
    arr = np.array(img).astype(np.float32)
    # Sample noise std matching real distribution (roughly log-normal feel)
    # 60% mild (2  18 ), 40% moderate-heavy (18  40 )
```

```
    if random.random() < 0.60:
        std = random.uniform(2, 18)
    else:
        std = random.uniform(18, 40)
    noise = np.random.normal(0, std, arr.shape)
    arr   = np.clip(arr + noise, 0, 255).astype(np.uint8)
    return Image.fromarray(arr)


# ============================================================================
# CALIBRATED JPEG COMPRESSION
# Real: blockiness p25=7.7, p75=23.3     lower the quality floor
# ============================================================================

def apply_jpeg_compression(img):
    import io
    # Real blockiness suggests quality often in 30  75   range
    # Mixture: 23.6% clean (high quality), 62.6% mild, 21.5% heavy
    r = random.random()
    if r < 0.236:
        quality = random.randint(80, 97)   # clean
    elif r < 0.862:
        quality = random.randint(45, 79)   # mild degradation
    else:
        quality = random.randint(20, 44)   # heavy degradation
    buf = io.BytesIO()
    img.save(buf, format='JPEG', quality=quality)
    buf.seek(0)
    return Image.open(buf).copy()


# ============================================================================
# GRAYSCALE CONVERSION
# Real: 23.1% of images are grayscale     add this step
# ============================================================================

def apply_grayscale(img):
    gray = img.convert('L')
    return gray.convert('RGB')   # keep 3 channels for consistency


# ============================================================================
# CALIBRATED AUGMENTATION PIPELINE
# Probabilities recalibrated to match real dataset statistics
# ============================================================================

def augment(img, bg_color, font_path):
    # Neighboring text bleed (unchanged probability)
    img = add_text_bleed(img, font_path, bg_color)

    # Bars / scan artifacts (unchanged)
    if random.random() < 0.3:
        img = add_bars(img, bg_color)

    # Rotation     now uses calibrated distribution
    img = apply_rotation(img)

    # Perspective     unchanged probability
    if random.random() < 0.7:
```

```
        img = apply_perspective(img)

    # Blur     reduced from 80% to 13.3% to match real data
    img = apply_blur(img)

    # Noise     always apply (real images all have some noise); std calibrated
    img = apply_noise(img)

    # JPEG compression     always apply; quality levels calibrated
    img = apply_jpeg_compression(img)

    # Grayscale     23.1% of real images are grayscale
    if random.random() < 0.231:
        img = apply_grayscale(img)

    return img


# ============================================================================
# GENERATE
# ============================================================================

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
IMAGES_DIR = OUTPUT_DIR / 'images'
IMAGES_DIR.mkdir(exist_ok=True)

metadata      = []
img_counter   = 0
layout_counts = {'straight': 0, 'circular': 0, 'sinusoidal': 0}

print(f"\n{'='*60}")
print(f"GENERATING {NUM_IMAGES} CALIBRATED SYNTHETIC SCENE TEXT IMAGES")
print(f"  ~{LAYOUT_CIRCULAR*100:.0f}% circular  |  "
      f"~{LAYOUT_SINUSOID*100:.0f}% sinusoidal  |  "
      f"~{LAYOUT_STRAIGHT*100:.0f}% straight")
print(f"{'='*60}")

while img_counter < NUM_IMAGES:
    word      = random.choice(WORDS)
    font_path = random.choice(VALID_FONTS)
    font_name = Path(font_path).stem
    bg, fg    = pick_bg_and_text_color()

    try:
        img, layout = render_word(word, font_path, bg, fg)
        img = augment(img, bg, font_path)

        img_fname = f"img_{img_counter:06d}.jpg"
        img.save(str(IMAGES_DIR / img_fname), 'JPEG', quality=92)

        metadata.append({
            'image_id':   img_counter,
            'image_file': img_fname,
            'image_path': str(IMAGES_DIR / img_fname),
            'label':      word,
            'font':       font_name,
            'layout':     layout,
            'bg_color':   str(bg),
            'text_color': str(fg),
```

```
        })

        layout_counts[layout] += 1
        img_counter += 1

        if img_counter % 1000 == 0:
            print(f"  [{img_counter}/{NUM_IMAGES}] "
                  f"straight={layout_counts['straight']} "
                  f"circular={layout_counts['circular']} "
                  f"sinusoidal={layout_counts['sinusoidal']}")

    except Exception:
        continue

print(f"\ n   {img_counter} images generated")
print(f"  Layout breakdown:")
for k, v in layout_counts.items():
    print(f"    {k:>12} : {v:>5}  ({v/img_counter*100:.1f}%)")

# ============================================================================
# METADATA
# ============================================================================

with open(OUTPUT_DIR / 'metadata.csv', 'w', encoding='utf-8', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=metadata[0].keys())
    writer.writeheader()
    writer.writerows(metadata)

with open(OUTPUT_DIR / 'metadata.json', 'w', encoding='utf-8') as f:
    json.dump(metadata, f, ensure_ascii=False, indent=2)

import pandas as pd
df_synth = pd.DataFrame([{
    'image_path':   m['image_path'],
    'label':        m['label'],
    'layout':       m['layout'],
    'is_synthetic': True,
} for m in metadata])
df_synth.to_csv(OUTPUT_DIR / 'synth_dataset.csv', index=False, encoding='utf-8')

print(f"\n{'='*60}")
print(f"DONE")
print(f"{'='*60}")
print(f"  Total images : {img_counter}")
print(f"  Unique words : {len(set(m['label'] for m in metadata))}")
print(f"  Fonts used   : {len(set(m['font'] for m in metadata))}")
print(f"  Output dir   : {OUTPUT_DIR}")



"""
Synthetic Scene Text     Visualization (Kaggle-compatible Bengali labels)
=======================================
Bengali fonts often fail to render in matplotlib on Kaggle.
This version shows image index numbers on the grid and prints
the Bengali labels as a separate text table.
"""

```

```
819 import pandas as pd
820 import matplotlib.pyplot as plt
821 from PIL import Image
822 from pathlib import Path
823 import math
824
825 # ================================================================================
826 # CONFIGURATION
827 # ================================================================================
828
829 OUTPUT_DIR = Path('/kaggle/working/synthetic_scene_text')
830 SYNTH_CSV  = OUTPUT_DIR / 'synth_dataset.csv'
831
832 COLS    = 5
833 ROWS    = 8   # 40 images shown
834 FIGSIZE = (20, 28)
835
836 # ================================================================================
837 # LOAD METADATA
838 # ================================================================================
839
840 df = pd.read_csv(SYNTH_CSV)
841 print(f"    Loaded {len(df)} images from {SYNTH_CSV}")
842 print(f"  Unique words : {df['label'].nunique()}")
843 print(f"\nSample rows:")
844 print(df.head())
845
846 # ================================================================================
847 # VISUALIZATION 1     Grid with INDEX numbers instead of Bengali labels
848 # ================================================================================
849
850 sample = df.sample(min(ROWS * COLS, len(df)), random_state=42).reset_index(drop=True)
851
852 fig, axes = plt.subplots(ROWS, COLS, figsize=FIGSIZE)
853 fig.patch.set_facecolor('#1a1a2e')
854
855 for idx, ax in enumerate(axes.flat):
856     if idx >= len(sample):
857         ax.axis('off')
858         continue
859
860     row      = sample.iloc[idx]
861     img_path = row['image_path']
862
863     try:
864         img = Image.open(img_path)
865         ax.imshow(img)
866     except Exception:
867         ax.set_facecolor('#333')
868         ax.text(0.5, 0.5, 'Load error', ha='center', va='center',
869                 color='red', transform=ax.transAxes)
870
871     # Use numeric index     renders perfectly without Bengali font
872     ax.set_xlabel(f'#{idx}', fontsize=12, color='#f0e68c',
873                   labelpad=4, fontweight='bold')
874     ax.set_xticks([])
875     ax.set_yticks([])
876     for spine in ax.spines.values():
877         spine.set_edgecolor('#444')
```

```
        spine.set_linewidth(1.5)

plt.suptitle('Synthetic Bengali Scene Text     Sample Grid  (see label table below)',
             fontsize=16, fontweight='bold', color='white', y=1.005)
plt.tight_layout(pad=1.5)

out1 = OUTPUT_DIR / 'viz_sample_grid.png'
plt.savefig(out1, dpi=120, bbox_inches='tight', facecolor='#1a1a2e')
plt.show()
print(f"\ n   Grid saved: {out1}")

# ============================================================================
# LABEL TABLE     printed separately (bypasses font rendering entirely)
# ============================================================================

print("\n" + "="*55)
print("LABEL TABLE  (#index     Bengali word)")
print("="*55)
col_width = 4  # how many columns to print side by side
rows_text = []
for idx, row in sample.iterrows():
    rows_text.append(f"#{idx:<3}  {row['label']}")

# Print in columns
per_col = math.ceil(len(rows_text) / col_width)
for r in range(per_col):
    line = "    ".join(
        rows_text[r + c * per_col]
        for c in range(col_width)
        if r + c * per_col < len(rows_text)
    )
    print(line)

# Also save as a plain-text file for easy reference
label_txt = OUTPUT_DIR / 'viz_sample_labels.txt'
with open(label_txt, 'w', encoding='utf-8') as f:
    f.write("# Label Table     matches grid indices in viz_sample_grid.png\n")
    f.write("index\tlabel\timage_path\n")
    for idx, row in sample.iterrows():
        f.write(f"#{idx}\t{row['label']}\t{row['image_path']}\n")
print(f"\ n   Label table saved: {label_txt}")

# ============================================================================
# VISUALIZATION 2     Augmentation variety (same word, different augmentations)
# ============================================================================

word_counts  = df['label'].value_counts()
common_words = word_counts[word_counts >= 4].index.tolist()

if common_words:
    pick_word    = common_words[0]
    word_samples = df[df['label'] == pick_word].head(10)

    n    = len(word_samples)
    ncols = math.ceil(n / 2)
    fig2, axes2 = plt.subplots(2, ncols, figsize=(20, 8))
    fig2.patch.set_facecolor('#0f3460')
    axes2_flat = axes2.flat

```

```
    for idx, (_, row) in enumerate(word_samples.iterrows()):
        ax = axes2_flat[idx]
        try:
            img = Image.open(row['image_path'])
            ax.imshow(img)
        except Exception:
            pass
        ax.set_title(f"#{idx}", fontsize=11, color='#e94560',
                     fontweight='bold', pad=4)  # index only, no Bengali
        ax.axis('off')

    for ax in list(axes2_flat)[n:]:
        ax.axis('off')

    # Print the word in stdout instead
    print(f"\ n   Augmentation variety plot     word: {pick_word!r}")
    plt.suptitle("Same Word     Augmentation Variety  (label printed below)",
                 fontsize=15, fontweight='bold', color='white', y=1.02)
    plt.tight_layout()
    out2 = OUTPUT_DIR / 'viz_augmentation_variety.png'
    plt.savefig(out2, dpi=120, bbox_inches='tight', facecolor='#0f3460')
    plt.show()
    print(f"    Augmentation variety saved: {out2}")
else:
    print("    Not enough repeated words for augmentation variety plot")

# ================================================================================
# VISUALIZATION 3     Stats (no Bengali text, just character counts & fonts)
# ================================================================================

fig3, axes3 = plt.subplots(1, 3, figsize=(18, 5))
fig3.patch.set_facecolor('#16213e')

word_lengths = df['label'].str.len()

# Word length distribution
axes3[0].hist(word_lengths, bins=range(1, word_lengths.max() + 2),
              color='#e94560', edgecolor='white', alpha=0.85)
axes3[0].set_title('Word Length Distribution', color='white', fontsize=13, pad=10)
axes3[0].set_xlabel('Number of Characters', color='#aaa')
axes3[0].set_ylabel('Count', color='#aaa')
axes3[0].set_facecolor('#1a1a2e')
axes3[0].tick_params(colors='white')
for spine in axes3[0].spines.values():
    spine.set_edgecolor('#444')

# Top 15 words     show character length as label, not Bengali text
top_words = word_counts.head(15)
short_labels = [f"len={len(w)}" for w in top_words.index]  # safe ASCII label
axes3[1].barh(range(len(top_words)), top_words.values,
              color='#0f3460', edgecolor='#e94560', linewidth=1.2)
axes3[1].set_yticks(range(len(top_words)))
axes3[1].set_yticklabels(short_labels, fontsize=10, color='#f0e68c')
axes3[1].set_title('Top 15 Words (by freq)', color='white', fontsize=13, pad=10)
axes3[1].set_xlabel('Frequency', color='#aaa')
axes3[1].set_facecolor('#1a1a2e')
axes3[1].tick_params(colors='white')
for spine in axes3[1].spines.values():
    spine.set_edgecolor('#444')
```

```
# Font distribution
font_counts = df['font'].value_counts().head(15)
axes3[2].barh(range(len(font_counts)), font_counts.values,
              color='#533483', edgecolor='#e94560', linewidth=1.2)
axes3[2].set_yticks(range(len(font_counts)))
axes3[2].set_yticklabels(font_counts.index, fontsize=9, color='#f0e68c')
axes3[2].set_title('Font Usage (Top 15)', color='white', fontsize=13, pad=10)
axes3[2].set_xlabel('Count', color='#aaa')
axes3[2].set_facecolor('#1a1a2e')
axes3[2].tick_params(colors='white')
for spine in axes3[2].spines.values():
    spine.set_edgecolor('#444')

plt.suptitle('Synthetic Dataset Statistics',
             fontsize=16, fontweight='bold', color='white', y=1.02)
plt.tight_layout()
out3 = OUTPUT_DIR / 'viz_stats.png'
plt.savefig(out3, dpi=120, bbox_inches='tight', facecolor='#16213e')
plt.show()
print(f"    Stats saved: {out3}")

# ============================================================================
# SUMMARY
# ============================================================================

print(f"\n{'='*55}")
print(f"VISUALIZATION COMPLETE")
print(f"{'='*55}")
print(f"  Total images     : {len(df)}")
print(f"  Unique words     : {df['label'].nunique()}")
print(f"  Unique fonts     : {df['font'].nunique()}")
print(f"  Avg word length  : {word_lengths.mean():.1f} chars")
print(f"\nSaved:")
print(f"  {out1.name}      sample grid (indices)")
print(f"  {label_txt.name}      Bengali label table")
if common_words:
    print(f"  {out2.name}      augmentation variety")
print(f"  {out3.name}      dataset statistics")
```

Generated using Undegraduate Thesis LaTeX Template, Version 2.0. Department of Electrical and Electronic Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh.

This thesis was generated on Sunday 7th June, 2026 at 2:07pm.

B.Sc.

Engg.

EEE

BUET

PATHOK: A Bangla Handwritten and Scene Text Recognition Model using Grapheme Tokenization

MI Sajin

MK Islam

June

2026